LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG TRONG PYTHON



MỤC LỤC Trang

1. Tìm hiểu mọi thứ về Class trong Python3 1
1.1. Class 1
1.2. Method – Phương thức 2
1.3. Kế thừa 3
1.4. Iterators 5
1.5. Generators 6
2. Lập trình hướng đối tượng trong Python | Phần 1 (lớp, đối tượng và các thành viên của lớp) 7
2.1. Tham số self 7
2.2. Phương thức __init__ 8
2.3. Instance variable và Class variable 9
2.4. Làm thế nào để tạo ra một class trống? 10
3. Lập trình hướng đối tượng trong Python | Phần 2 (che giấu dữ liệu và in đối tượng) 11
3.1. Che giấu dữ liệu 11
3.2. In ra các đối tượng 12
4. Lập trình hướng đối tượng trong Python | Phần 3 (Kế thừa, ví dụ về đối tượng, hàm issubclass() và super()) 14
4.1. Làm thế nào để kiểm tra xem một lớp có phải là lớp con của lớp khác không 15
4.2. Object class – lớp đối tượng là gì? 16
4.3. Python có hỗ trợ đã kế thừa – multiple inheritance không? 16
4.4. Làm thế nào để truy cập tới các thành viên của lớp cha, tại lớp con? 17
5. Đa hình trong Python 19
5.1. Ví dụ về các hàm có tính đa hình được tích hợp sẵn trong Python 19
5.2. Ví dụ về hàm có tính đa hình do người dùng tự định nghĩa 20
5.3. Đa hình với các phương thức của lớp 20
5.4. Đa hình và Kế thừa 21
5.5. Đa hình đối với một hàm và nhiều đối tượng 22
6. Class variable và Static variable trong Python 23
7. Class method và static method trong Python 25
7.1. Class Method – Phương thức thuộc về lớp 25
7.2. Static method – Phương thức tĩnh 26
7.3. Class method và Static method 26
7.4. Khi nào nên sử dụng cái nào? 26
7.5. Làm thế nào để khai báo class method và static method? 27

8. Thay đổi các biến thành viên trong Python 28
9. Hàm constructor trong Python 30
10. Hàm destructor trong Python 32
11. Hàm str() và hàm repr() trong Python 35
12. Metaprogramming bằng các Meta-classes trong Python 38
12.1. Metaclass 38
12.2. Tạo ra các custom Metaclass – Metaclass tùy chỉnh 41
12.3. Sử dụng metaclass để giải quyết các bài toán 43
12.4. Khi nào nên sử dụng Metaclass 46
13. Các thuộc tính của Lớp và Các thuộc tính của Thể hiện trong Python 47
13.1. Các thuộc tính của Lớp 47
13.2. Các thuộc tính của thể hiện 48
14. Reflection trong Python 49
14.1. Reflection-enabling functions 50
14.2. Hàm Callable() 55
14.3. Dir: 56
14.4. Getattr 56
15. Barrier Object trong Python 57
15.1. Khởi tạo một barrier 57
15.2. Kiểm tra trạng thái của barrier: 59
15.3. parties: Số lượng các threads cần phải truyền vào cho barrier. 59
15.4. Hủy bỏ một barrier: 59
15.5. Khởi động lại barrier: 59
15.6. wait: Vượt qua barrier – rào cản. 59
16. Timer objects trong Python 61
16.1. Tạo ra một đối tượng Timer 61
16.2. Hủy đi một timer 62
17. Garbage Collection trong Python 62
17.1. Các cách để tạo ra một đối tượng mà garbage collector có thể thu gom 64
17.2. Việc tự động thu hồi bộ nhớ đối với các chu trình tham chiếu 64
17.3. Thu hồi bộ nhớ thủ công 65
18. Bài tập hướng đối tượng trong python 67

1.Tìm hiểu mọi thứ về Class trong Python3

Sau khi đã trải qua các bài học về Python cơ bản, có lẽ lúc này đây bạn đang rất mong chờ để tìm hiểu tiếp, để được biết nhiều hơn về các kiến thức nâng cao của ngôn ngữ lập trình Python3. Bài học này sẽ là một sự khởi đầu để đưa các bạn đến với các kiến thức trung cấp về Python3.

Để đi sâu hơn về class và lập trình hướng đối tượng trong python, bạn có thể tham khảo phần 7 của series tự học python.

Hãy thực hành thường xuyên để nhớ được kiến thức một cách sâu sắc, và đừng quên rằng bạn có thể sử dụng phím Tab để thụt lề cho các dòng code Python của mình, điều này sẽ giúp code của bạn trở nên ngăn nắp hơn.

Bài này sẽ được chia thành 5 phần sau:

Nội dung chính

1. Class
2. Method – Phương thức
3. Kế thừa
4. Iterators
5. Generators

1.1.Class

Cũng giống như các ngôn ngữ lập trình hướng đối tượng khác, Python có hỗ trợ các class. Dưới đây là một vài điểm cơ bản về class trong Python:

– Các class đều được tạo ra bằng từ khóa class.

– Các thuộc tính chính là các biến thuộc về class.

– Các thuộc tính thường là public, và có thể được truy cập tới bằng cách sử dụng toán tử dấu chấm (.), ví dụ: Myclass.Myattribute.

Ví dụ về class:

```
# -----------------------------------------------------------
#Cafedev.vn - Kênh thông tin IT hàng đầu Việt Nam
#@author cafedevn
#Contact: cafedevn@gmail.com
#Fanpage: https://www.facebook.com/cafedevn
#Instagram: https://instagram.com/cafedevn
#Twitter: https://twitter.com/CafedeVn
#Linkedin: https://www.linkedin.com/in/cafe-dev-407054199/
# -----------------------------------------------------------
# creates a class named MyClass
class MyClass:
```

```
        # assign the values to the MyClass attributes
        number = 0
        name = "noname"
def Main():
        # Creating an object of the MyClass.
        # Here, 'me' is the object
        me = MyClass()
        # Accessing the attributes of MyClass
        # using the dot(.) operator
        me.number = 1337
        me.name = "Harssh"
        # str is an build-in function that
        # creates an string
        print(me.name + " " + str(me.number))
# telling python that there is main in the program.
if __name__=='__main__':
        Main()
```

Kết quả in ra là:

```
Harssh 1337
```

1.2. Method – Phương thức

Phương thức là một nhóm code có chức năng thực hiện một tác vụ cụ thể bên trong chương trình Python của bạn.

– Hàm mà thuộc về một class cụ thể thì được gọi là Phương thức.

– Tất cả các phương thức đều yêu cầu tham số 'self'. Nếu bạn đã từng code trên các ngôn ngữ OOP khác, thì 'self' trong Python chính là 'this' ở trong các ngôn ngữ đó. 'self' và 'this' đều mang ý nghĩa là một đối tượng hiện tại của một class nào đó.

– Để tạo ra một phương thức mới, ta sử dụng từ khóa 'def'.

```
# ------------------------------------------------------------
#Cafedev.vn - Kênh thông tin IT hàng đầu Việt Nam
#@author cafedevn
#Contact: cafedevn@gmail.com
#Fanpage: https://www.facebook.com/cafedevn
#Instagram: https://instagram.com/cafedevn
#Twitter: https://twitter.com/CafedeVn
#Linkedin: https://www.linkedin.com/in/cafe-dev-407054199/
```

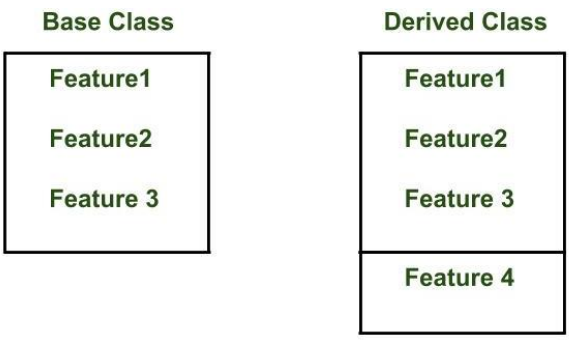
```
# ----------------------------------------------------------
# A Python program to demonstrate working of class
# methods
class Vector2D:
        x = 0.0
        y = 0.0
        # Creating a method named Set
        def Set(self, x, y):
                self.x = x
                self.y = y
def Main():
        # vec is an object of class Vector2D
        vec = Vector2D()
        # Passing values to the function Set
        # by using dot(.) operator.
        vec.Set(5, 6)
        print("X: " + str(vec.x) + ", Y: " + str(vec.y))
if __name__=='__main__':
        Main()
```

Kết quả in ra là:

```
X: 5, Y: 6
```

1.3.Kế thừa

Kế thừa trong lập trình OOP được định nghĩa là cách mà một class cụ thể sẽ kế thừa lại các tính năng từ một base class – tức là lớp cha/lớp cơ sở. Base class còn được gọi là 'Superclass', và class mà kế thừa từ Superclass thì được gọi là 'Subclass' tức là lớp con.



Inheritance

Từ hình vẽ trên ta thấy rằng class Derived có thể kế thừa các tính năng từ class Base, đồng thời nó cũng có thể tự định nghĩa ra các tính năng của riêng mình.

Ví dụ về cú pháp thực hiện kế thừa trong Python:

```
class derived-classname(superclass-name)
```

Ví dụ mô tả cách thức hoạt động của kế thừa trong Python:

```
# A Python program to demonstrate working of inheritance
class Pet:
        #__init__ is an constructor in Python
        def __init__(self, name, age):
                self.name = name
                self.age = age
# Class Cat inheriting from the class Pet
class Cat(Pet):
        def __init__(self, name, age):
                # calling the super-class function __init__
                # using the super() function
                super().__init__(name, age)
def Main():
        thePet = Pet("Pet", 1)
        jess = Cat("Jess", 3)
        # isinstance() function to check whether a class is
        # inherited from another class
        print("Is jess a cat? " +str(isinstance(jess, Cat)))
        print("Is jess a pet? " +str(isinstance(jess, Pet)))
        print("Is the pet a cat? "+str(isinstance(thePet, Cat)))
        print("Is thePet a Pet? " +str(isinstance(thePet, Pet)))
        print(jess.name)
if __name__=='__main__':
        Main()
```

Kết quả in ra là:

```
Is jess a cat? True
Is jess a pet? True
Is the pet a cat? False
Is thePet a Pet? True
Jess
```

1.4.Iterators

Iterators là các đối tượng mà chúng ta có thể thực hiện các phép lặp/duyệt trên chúng.

– Python sử dụng phương thức __iter__() để trả về một đối tượng iterator của class

– Đối tượng iterator sau đó sẽ sử dụng phương thức __next__() để lấy được next item – phần tử tiếp theo.

– Vòng lặp for sẽ dừng lại khi ngoại lệ StopIteration Exception được kích hoạt.

Dưới đây là ví dụ đảo ngược một chuỗi, được thực hiện trong hàm main

```
# This program will reverse the string that is passed
# to it from the main function
class Reverse:
    def __init__(self, data):
        self.data = data
        self.index = len(data)
    def __iter__(self):
        return self
    def __next__(self):
        if self.index == 0:
            raise StopIteration
        self.index-= 1
        return self.data[self.index]
def Main():
    rev = Reverse('Drapsicle')
    for char in rev:
        print(char)
if __name__=='__main__':
    Main()
```

Kết quả in ra là:

```
e
l
c
i
s
p
a
```

```
r
D
```

1.5.Generators

– Sử dụng generator là một cách khác để tạo ra các iterators.

– Thay vì phải sử dụng một class riêng biệt, ta chỉ cần sử dụng tới một hàm.

– Tạo ra phần code cơ sở dành cho phương thức/hàm next() và iter()

– Sử dụng một câu lệnh đặc biệt được gọi là 'yield' giúp lưu lại trạng thái (state) của generator và thiết lập một điểm trở về – resume point nhằm dự phòng cho trường hợp hàm next() được gọi lại lần nữa.

Dưới đây là đoạn chương trình Python mô tả hoạt động của các Generators

```
# A Python program to demonstrate working of Generators
def Reverse(data):
    # this is like counting from 100 to 1 by taking one(-1)
    # step backward.
    for index in range(len(data)-1, -1, -1):
        yield data[index]
def Main():
    rev = Reverse('Harssh')
    for char in rev:
        print(char)
    data ='Harssh'
    print(list(data[i] for i in range(len(data)-1, -1, -1)))
if __name__=="__main__":
    Main()
```

Kết quả in ra là:

```
h
s
s
r
a
H
['h', 's', 's', 'r', 'a', 'H']
```

2.Lập trình hướng đối tượng trong Python | Phần 1 (lớp, đối tượng và các thành viên của lớp)

Dưới đây là một chương trình Python đơn giản, tạo ra một lớp có chứa 1 phương thức.

```
# ----------------------------------------------------------
#Cafedev.vn - Kênh thông tin IT hàng đầu Việt Nam
#@author cafedevn
#Contact: cafedevn@gmail.com
#Fanpage: https://www.facebook.com/cafedevn
#Instagram: https://instagram.com/cafedevn
#Twitter: https://twitter.com/CafedeVn
#Linkedin: https://www.linkedin.com/in/cafe-dev-407054199/
# ----------------------------------------------------------
# A simple example class
class Test:
    # A sample method
    def fun(self):
        print("Hello")
# Driver code
obj = Test()
obj.fun()
```

Kết quả in ra là:

```
Hello
```

Như bạn có thể thấy ở ví dụ trên, chúng ta đã tạo ra một lớp mới bằng cách sử dụng câu lệnh class cùng với một cái tên mình muốn đặt cho lớp đó. Tiếp theo là một khối các câu lệnh được thụt lề, tạo thành phần thân của lớp. Trong trường hợp này, chúng ta đã định nghĩa ra một phương thức duy nhất, ở trong một lớp.

Tiếp theo, chúng ta sẽ tạo ra một đối tượng/thể hiện của lớp này bằng cách sử dụng tên của class, theo sau là một cặp dấu ngoặc tròn.

Nội dung chính

- 1. Tham số self
- 2. Phương thức __init__
- 3. Instance variable và Class variable
- 4. Làm thế nào để tạo ra một class trống?

2.1.Tham số self

1. Các phương thức của lớp đều phải có thêm một tham số đầu tiên trong phần method definition – phần code định nghĩa phương thức. Chúng ta sẽ

không cung cấp giá trị nào cho tham số này khi gọi tới phương thức, mà Python sẽ lo phần việc này.

2. Giả sử chúng ta có một phương thức mà không nhận vào bất kỳ đối số (argument) nào, tuy nhiên, thực tế là phương thức đó vẫn sẽ có một đối số được ẩn đi, chính là self. Hãy xem hàm fun() trong ví dụ đơn giản ở trên.

3. self trong Python thì tương đương với con trỏ this trong C++ và tham chiếu this trong Java.

Khi chúng ta gọi đến một phương thức của một đối tượng cụ thể, ví dụ myobject.method(arg1, arg2), câu lệnh trên sẽ được Python tự động chuyển đổi thành Myclass.method(myobject, arg1, arg2) – Đây chính là bản chất đặc biệt của self trong Python.

2.2.Phương thức __init__

Phương thức __init__ tương tự với hàm constructor trong C++ và Java. Nó sẽ chạy ngay sau khi một đối tượng của một lớp được thể hiện. Phương thức này rất hữu dụng trong việc thực hiện các tác vụ khởi tạo cần thiết mà bạn muốn làm trên đối tượng của mình.

```
# A Sample class with init method
class Person:

    # init method or constructor
    def __init__(self, name):
        self.name = name

    # Sample Method
    def say_hi(self):
        print('Hello, my name is', self.name)

p = Person('cafedev')
p.say_hi()
```

Kết quả in ra là:

```
Hello, my name is cafedev
```

Trong ví dụ này, chúng ta đã định nghĩa cho phương thức __init__ nhận vào một tham số tên là name (đi kèm với một tham số self thông thường).

2.3.Instance variable và Class variable

Trong Python, Instance variable là các biến mà giá trị của chúng được gán ở bên trong một hàm constructor hoặc ở bên trong một phương thức, bằng từ khóa self.

Class variable là các biến mà giá trị của chúng được gán ở bên trong lớp (class).

Dưới đây là đoạn chương trình Python mô tả các biến mà có giá trị được gán ở bên trong phần code định nghĩa class thì được gọi là các Class variables, còn các biến được gán giá trị ở bên trong các phương thức và hàm constructor thì là các Instance variable.

```
# -----------------------------------------------------------
#Cafedev.vn - Kênh thông tin IT hàng đầu Việt Nam
#@author cafedevn
#Contact: cafedevn@gmail.com
#Fanpage: https://www.facebook.com/cafedevn
#Instagram: https://instagram.com/cafedevn
#Twitter: https://twitter.com/CafedeVn
#Linkedin: https://www.linkedin.com/in/cafe-dev-407054199/
# -----------------------------------------------------------
# Python program to show that the variables with a value
# assigned in class declaration, are class variables and
# variables inside methods and constructors are instance
# variables.
# Class for Computer Science Student
class CSStudent:
    # Class Variable
    stream = 'cse'
    # The init method or constructor
    def __init__(self, roll):
        # Instance Variable
        self.roll = roll
# Objects of CSStudent class
a = CSStudent(101)
b = CSStudent(102)
print(a.stream)  # prints "cse"
print(b.stream)  # prints "cse"
print(a.roll)    # prints 101
# Class variables can be accessed using class
```

```
# name also
print(CSStudent.stream) # prints "cse"
```

Chúng ta cũng có thể định nghĩa các instance variables bên trong các phương thức bình thường

```
# Python program to show that we can create
# instance variables inside methods
# Class for Computer Science Student
class CSStudent:
    # Class Variable
    stream = 'cse'
    # The init method or constructor
    def __init__(self, roll):
        # Instance Variable
        self.roll = roll
    # Adds an instance variable
    def setAddress(self, address):
        self.address = address
    # Retrieves instance variable
    def getAddress(self):
        return self.address
# Driver Code
a = CSStudent(101)
a.setAddress("Noida, UP")
print(a.getAddress())
```

Kết quả in ra là:

```
Noida, UP
```

2.4.Làm thế nào để tạo ra một class trống?

Chúng ta có thể tạo ra một class trống (empty class) bằng cách sử dụng câu lệnh pass trong Python

```
# An empty class
class Test:
    pass
```

Nguồn và Tài liệu tiếng anh tham khảo:

3.Lập trình hướng đối tượng trong Python | Phần 2 (che giấu dữ liệu và in đối tượng)

3.1.Che giấu dữ liệu

Trong Python, chúng ta sử dụng hai dấu gạch dưới liên tiếp (tức là __) trước tên của các thuộc tính, và các thuộc tính đó sẽ không thể được nhìn thấy trực tiếp, khi ở bên ngoài lớp.

Dưới đây là một ví dụ về che giấu dữ liệu của lớp trong Python:

```
# -----------------------------------------------------------
#Cafedev.vn - Kênh thông tin IT hàng đầu Việt Nam
#@author cafedevn
#Contact: cafedevn@gmail.com
#Fanpage: https://www.facebook.com/cafedevn
#Instagram: https://instagram.com/cafedevn
#Twitter: https://twitter.com/CafedeVn
#Linkedin: https://www.linkedin.com/in/cafe-dev-407054199/
# -----------------------------------------------------------
class MyClass:
    # Hidden member of MyClass
    __hiddenVariable = 0
    # A member method that changes
    # __hiddenVariable
    def add(self, increment):
        self.__hiddenVariable += increment
        print (self.__hiddenVariable)
# Driver code
myObject = MyClass()
myObject.add(2)
myObject.add(5)
# This line causes error
print (myObject.__hiddenVariable)
```

Kết quả in ra là:

```
2
7
Traceback (most recent call last):
  File "filename.py", line 13, in
```

```
    print (myObject.__hiddenVariable)
AttributeError: MyClass instance has
no attribute '__hiddenVariable'
```

Trong đoạn chương trình trên, chúng ta đã cố truy cập tới biến được che giấu, từ bên ngoài lớp bằng cách sử dụng đối tượng myObject, và một ngoại lệ (exception) đã được ném ra.

Chúng ta có thể truy cập đến giá trị của thuộc tính được che giấu bằng một cú pháp mẹo sau:

Đoạn chương trình Python mô tả cách mà các thành viên được che giấu có thể được truy cập đến, từ bên ngoài lớp:

```
# A Python program to demonstrate that hidden
# members can be accessed outside a class
class MyClass:
    # Hidden member of MyClass
    __hiddenVariable = 10
# Driver code
myObject = MyClass()
print(myObject._MyClass__hiddenVariable)
```

Kết quả in ra là:

```
10
```

Các private methods – phương thức bí mật đều có thể được truy cập tới từ bên ngoài lớp mà chúng thuộc về, chỉ là việc truy cập này sẽ khó khăn hơn một chút. Trong Python thì không có cái gì thực sự là private; về bản chất, tên của các phương thức bí mật và thuộc tính bí mật sẽ được Python xử lý sao cho chúng trông như không thể được truy cập tới thông qua những cái tên đó.

3.2.In ra các đối tượng

Việc in ra các đối tượng cung cấp cho chúng ta các thông tin về những đối tượng mà mình đang phải làm việc cùng. Trong C++, chúng ta có thể thực hiện điều này bằng cách thêm một hàm bạn ostream& operator << (ostream&, const Foobar&) cho lớp. Trong Java, chúng ta sẽ sử dụng phương thức toString(). Trong Python, có thể đạt được điều này bằng cách sử dụng các phương thức __repr__or__str.

```
class Test:
    def __init__(self, a, b):
        self.a = a
        self.b = b
    def __repr__(self):
        return "Test a:%s b:%s" % (self.a, self.b)
    def __str__(self):
```

```
        return "From str method of Test: a is %s," \
               "b is %s" % (self.a, self.b)
# Driver Code
t = Test(1234, 5678)
print(t) # This calls __str__()
print([t]) # This calls __repr__()
```

Kết quả in ra là:

```
From str method of Test: a is 1234,b is 5678
[Test a:1234 b:5678]
```

Một số điểm quan trọng khi in ra đối tượng

– Nếu không có phương thức __str__ nào được định nghĩa, thì hàm print t (hoặc print str(t)) sẽ sử dụng __repr__

```
class Test:
    def __init__(self, a, b):
        self.a = a
        self.b = b
    def __repr__(self):
        return "Test a:%s b:%s" % (self.a, self.b)
# Driver Code
t = Test(1234, 5678)
print(t)
```

Kết quả in ra là:

```
Test a:1234 b:5678
```

– Nếu không có phương thức __repr__ nào được định nghĩa, thì phương thức mặc định sẽ được sử dụng.

```
class Test:
    def __init__(self, a, b):
        self.a = a
        self.b = b
# Driver Code
t = Test(1234, 5678)
print(t)
```

Kết quả in ra là:

```
<__main__.Test instance at 0x7fa079da6710>
```

4.Lập trình hướng đối tượng trong Python | Phần 3 (Kế thừa, ví dụ về đối tượng, hàm issubclass() và super())

Cho đến nay, những chủ đề về lập trình hướng đối tượng trong Python mà chúng ta đã thảo luận là:

– Phần 1 (lớp, đối tượng và các thành viên của lớp)

– Phần 2 (che giấu dữ liệu và in đối tượng)

Trong bài học này, tính kế thừa sẽ được giới thiệu.

Một trong các lợi thế chính của lập trình hướng đối tượng đó là khả năng tái sử dụng của code. Và kế thừa chính là một trong các cơ chế để đạt được điều đó. Trong kế thừa, một lớp (thường được gọi là lớp cha – super class) sẽ được kế thừa lại bởi một lớp khác (thường gọi là lớp con – subclass). Lớp con này sẽ bổ sung thêm một số thuộc tính cho lớp cha.

Dưới đây là một đoạn chương trình được viết bằng Python, mô tả cách cài đặt kế thừa trong Python

```
# ---------------------------------------------------------
#Cafedev.vn - Kênh thông tin IT hàng đầu Việt Nam
#@author cafedevn
#Contact: cafedevn@gmail.com
#Fanpage: https://www.facebook.com/cafedevn
#Instagram: https://instagram.com/cafedevn
#Twitter: https://twitter.com/CafedeVn
#Linkedin: https://www.linkedin.com/in/cafe-dev-407054199/
# ---------------------------------------------------------
# A Python program to demonstrate inheritance
# Base or Super class. Note object in bracket.
# (Generally, object is made ancestor of all classes)
# In Python 3.x "class Person" is
# equivalent to "class Person(object)"
class Person(object):
    # Constructor
    def __init__(self, name):
        self.name = name
    # To get name
    def getName(self):
```

```
        return self.name
    # To check if this person is employee
    def isEmployee(self):
        return False
# Inherited or Sub class (Note Person in bracket)
class Employee(Person):
    # Here we return true
    def isEmployee(self):
        return True
# Driver code
emp = Person("Geek1")  # An Object of Person
print(emp.getName(), emp.isEmployee())
emp = Employee("Geek2") # An Object of Employee
print(emp.getName(), emp.isEmployee())
```

Kết quả in ra là:

```
('Geek1', False)
('Geek2', True)
```

Nội dung chính

- 1. Làm thế nào để kiểm tra xem một lớp có phải là lớp con của lớp khác không
- 2. Object class – lớp đối tượng là gì?
- 3. Python có hỗ trợ đã kế thừa – multiple inheritance không?
- 4. Làm thế nào để truy cập tới các thành viên của lớp cha, tại lớp con?

4.1.Làm thế nào để kiểm tra xem một lớp có phải là lớp con của lớp khác không

Python có cung cấp một hàm có tên là issubclass(), có thể trực tiếp nói cho lập trình viên biết rằng lớp được xét này có phải là lớp con của một lớp khác hay không.

```
# Python example to check if a class is
# subclass of another
class Base(object):
    pass    # Empty Class
class Derived(Base):
    pass    # Empty Class
# Driver Code
print(issubclass(Derived, Base))
print(issubclass(Base, Derived))
```

```
d = Derived()
b = Base()
# b is not an instance of Derived
print(isinstance(b, Derived))
# But d is an instance of Base
print(isinstance(d, Base))
```

Kết quả in ra là:

```
True
False
False
True
```

4.2.Object class – lớp đối tượng là gì?

Cũng giống như lớp đối tượng trong Java, bắt đầu từ phiên bản 3.x của Python, đối tượng là gốc rễ của tất cả các lớp.

Trong Python 3.x, "class Test(object)" và "class Test" là như nhau.

Trong Python 2.x, "class Test(object)" sẽ tạo ra một lớp có đối tượng object làm lớp cha (được gọi là new style class), còn "class Test" sẽ tạo ra một old style class – lớp bình thường (không có đối tượng nào làm lớp cha của nó)

4.3.Python có hỗ trợ đã kế thừa – multiple inheritance không?

Không giống như Java và giống như C++, Python có hỗ trợ đã kế thừa. Chúng ta có thể chỉ định nhiều lớp cha, phân cách nhau bởi dấu phẩy, và tất cả được bao trong một cặp dấu ngoặc tròn () để thực hiện đa kế thừa.

Dưới đây là ví dụ mô tả cách thức hoạt động của đa kế thừa trong Python

```
# Python example to show working of multiple
# inheritance
class Base1(object):
    def __init__(self):
        self.str1 = "Cafedev1"
        print "Base1"
class Base2(object):
    def __init__(self):
        self.str2 = "Cafedev2"
        print "Base2"
class Derived(Base1, Base2):
    def __init__(self):
        # Calling constructors of Base1
```

```
        # and Base2 classes
        Base1.__init__(self)
        Base2.__init__(self)
        print "Derived"
    def printStrs(self):
        print(self.str1, self.str2)
ob = Derived()
ob.printStrs()
```

Kết quả in ra là:

```
Cafedev1
Cafedev2
Derived
('Cafedev1', 'Cafedev2')
```

4.4.Làm thế nào để truy cập tới các thành viên của lớp cha, tại lớp con?

1. Sử dụng tên lớp cha

Đoạn code ví dụ cho thấy các thành viên của lớp cha có thể được truy cập tới từ bên trong các lớp con, bằng cách sử dụng tên của lớp cha.

```
# Python example to show that base
# class members can be accessed in
# derived class using base class name
class Base(object):
    # Constructor
    def __init__(self, x):
        self.x = x
class Derived(Base):
    # Constructor
    def __init__(self, x, y):
        Base.x = x
        self.y = y
    def printXY(self):
       # print(self.x, self.y) will also work
       print(Base.x, self.y)
# Driver Code
d = Derived(10, 20)
d.printXY()
```

Kết quả in ra là:

```
(10, 20)
```

2. Sử dụng hàm super()

Chúng ta cũng có thể truy cập tới các thành viên của lớp cha bằng cách sử dụng hàm super().

Đoạn code dưới đây sẽ mô tả cách sử dụng hàm super() để truy cập tới các thành viên của lớp cha, từ lớp con

```
# ------------------------------------------------------------
#Cafedev.vn - Kênh thông tin IT hàng đầu Việt Nam
#@author cafedevn
#Contact: cafedevn@gmail.com
#Fanpage: https://www.facebook.com/cafedevn
#Instagram: https://instagram.com/cafedevn
#Twitter: https://twitter.com/CafedeVn
#Linkedin: https://www.linkedin.com/in/cafe-dev-407054199/
# ------------------------------------------------------------
# Python example to show that base
# class members can be accessed in
# derived class using super()
class Base(object):
    # Constructor
    def __init__(self, x):
        self.x = x
class Derived(Base):
    # Constructor
    def __init__(self, x, y):
        ''' In Python 3.x, "super().__init__(name)"
            also works'''
        super(Derived, self).__init__(x)
        self.y = y
    def printXY(self):
       # Note that Base.x won't work here
       # because super() is used in constructor
       print(self.x, self.y)
# Driver Code
d = Derived(10, 20)
```

```
d.printXY()
```

Kết quả in ra là:

```
(10, 20)
```

Lưu ý rằng, hai phương thức ở trên không hoàn toàn giống nhau. Trong bài học tiếp theo về kế thừa, chúng ta sẽ thảo luận tiếp hai chủ đề sau:

– Cách thức hoạt động của super? Làm thế nào mà việc truy cập tới thành viên (biến thành viên, hàm thành viên) của lớp cha, thông qua super và tên lớp cha lại khác nhau?

– Vấn đề Diamon (kim cương) được giải quyết thế nào trong Python?

5.Đa hình trong Python

Đa hình là gì: Từ đa hình (polymorphism) có nghĩa là sở hữu nhiều dạng thức/hình thức. Trong lập trình, đa hình có nghĩa là cùng một tên hàm (nhưng khác signatures) được sử dụng cho nhiều kiểu dữ liệu khác nhau.

Nội dung chính

- 1. Ví dụ về các hàm có tính đa hình được tích hợp sẵn trong Python
- 2. Ví dụ về hàm có tính đa hình do người dùng tự định nghĩa
- 3. Đa hình với các phương thức của lớp
- 4. Đa hình và Kế thừa
- 5. Đa hình đối với một hàm và nhiều đối tượng

5.1.Ví dụ về các hàm có tính đa hình được tích hợp sẵn trong Python

```
# ----------------------------------------------------------
#Cafedev.vn - Kênh thông tin IT hàng đầu Việt Nam
#@author cafedevn
#Contact: cafedevn@gmail.com
#Fanpage: https://www.facebook.com/cafedevn
#Instagram: https://instagram.com/cafedevn
#Twitter: https://twitter.com/CafedeVn
#Linkedin: https://www.linkedin.com/in/cafe-dev-407054199/
# ----------------------------------------------------------
# Python program to demonstrate in-built poly-
# morphic functions
# len() being used for a string
print(len("geeks"))
# len() being used for a list
print(len([10, 20, 30]))
```

Kết quả in ra là:

```
5
```

```
3
```

5.2.Ví dụ về hàm có tính đa hình do người dùng tự định nghĩa

```
# A simple Python function to demonstrate
# Polymorphism
def add(x, y, z = 0):
    return x + y+z
# Driver code
print(add(2, 3))
print(add(2, 3, 4))
```

Kết quả in ra là:

```
5
9
```

5.3.Đa hình với các phương thức của lớp

Đoạn code sau đây sẽ mô tả cách mà Python có thể sử dụng hai kiểu class khác nhau, theo cùng một cách. Chúng ta tạo ra một vòng lặp for để lặp qua một tuple chứa các đối tượng. Sau đó gọi đến các phương thức mong muốn, mà không cần phải lo về việc các đối tượng hiện tại thuộc kiểu class nào. Chúng ta sẽ giả sử rằng các phương thức này thực sự tồn tại bên trong mỗi lớp.

```
# -----------------------------------------------------------
#Cafedev.vn - Kênh thông tin IT hàng đầu Việt Nam
#@author cafedevn
#Contact: cafedevn@gmail.com
#Fanpage: https://www.facebook.com/cafedevn
#Instagram: https://instagram.com/cafedevn
#Twitter: https://twitter.com/CafedeVn
#Linkedin: https://www.linkedin.com/in/cafe-dev-407054199/
# -----------------------------------------------------------
class India():
    def capital(self):
        print("New Delhi is the capital of India.")
    def language(self):
        print("Hindi is the most widely spoken language of India.")
    def type(self):
        print("India is a developing country.")
class USA():
    def capital(self):
```

```
        print("Washington, D.C. is the capital of USA.")
    def language(self):
        print("English is the primary language of USA.")
    def type(self):
        print("USA is a developed country.")
obj_ind = India()
obj_usa = USA()
for country in (obj_ind, obj_usa):
    country.capital()
    country.language()
    country.type()
```

Kết quả in ra là

```
New Delhi is the capital of India.
Hindi is the most widely spoken language of India.
India is a developing country.
Washington, D.C. is the capital of USA.
English is the primary language of USA.
USA is a developed country.
```

5.4.Đa hình và Kế thừa

Trong Python, Đa hình cho phép chúng ta định nghĩa ra các phương thức trong lớp con mà có cùng tên với các phương thức trong lớp cha. Trong kế thừa, lớp con sẽ thừa kế các phương thức từ lớp cha. Tuy nhiên, ta hoàn toàn có thể chỉnh sửa các phương thức thuộc lớp con, mà được kế thừa lại từ lớp cha. Điều này đặc biệt hữ dụng trong những trường hợp mà phương thức được kế thừa lại từ lớp cha chưa thực sự phù hợp với lớp con. Trong những trường hợp như vậy, chúng ta sẽ cài đặt lại phương thức đó tại lớp con. Cái việc mà chúng ta cài đặt lại một phương thức được kế thừa từ lớp cha, tại lớp con, được gọi là ghi đè phương thức – Method Overriding.

```
class Bird:
  def intro(self):
    print("There are many types of birds.")
  def flight(self):
    print("Most of the birds can fly but some cannot.")
class sparrow(Bird):
  def flight(self):
    print("Sparrows can fly.")
class ostrich(Bird):
```

```
  def flight(self):
    print("Ostriches cannot fly.")
obj_bird = Bird()
obj_spr = sparrow()
obj_ost = ostrich()
obj_bird.intro()
obj_bird.flight()
obj_spr.intro()
obj_spr.flight()
obj_ost.intro()
obj_ost.flight()
```

Kết quả in ra là:

```
There are many types of birds.
Most of the birds can fly but some cannot.
There are many types of birds.
Sparrows can fly.
There are many types of birds.
Ostriches cannot fly.
```

5.5.Đa hình đối với một hàm và nhiều đối tượng

Chúng ta có thể tạo ra một hàm mà có khả năng nhận vào bất kỳ đối tượng thuộc kiểu class nào, nhằm thực hiện đa hình. Trong ví dụ ở mục này, chúng ta sẽ tạo ra một hàm có tên là 'func()' nhận vào một đối tượng tên là 'obj'. Mặc dù chúng ta đang sử dụng cái tên 'obj' cho đối tượng được truyền vào hàm 'func()', nhưng bất cứ đối tượng được thể hiện thuộc kiểu class nào cũng đều có thể được truyền vào cho tham số 'obj'. Tiếp theo, giao cho hàm này thực hiện một tác vụ nào đó mà có sử dụng tới đối tượng 'obj' mà chúng ta đã truyền vào. Trong trường hợp này, giả sử chúng ta gọi tới 3 phương thức là capital(), language() và type(), các phương thức này sẽ được định nghĩa trong class 'India' và class 'USA'. Tiếp theo, tạo ra hai thể hiện của lớp 'India' và lớp 'USA'. Với hai đối tượng vừa được thể hiện, chúng ta có thể khiến cho chúng "hành động" bằng cách cùng gọi đến hàm func():

Đoạn code sau sẽ cài đặt tính đa hình, chỉ bằng một hàm duy nhất

```
def func(obj):
    obj.capital()
    obj.language()
    obj.type()
obj_ind = India()
```

```
obj_usa = USA()
func(obj_ind)
func(obj_usa)
class India():
    def capital(self):
        print("New Delhi is the capital of India.")
    def language(self):
        print("Hindi is the most widely spoken language of India.")
    def type(self):
        print("India is a developing country.")
class USA():
    def capital(self):
        print("Washington, D.C. is the capital of USA.")
    def language(self):
        print("English is the primary language of USA.")
    def type(self):
        print("USA is a developed country.")
def func(obj):
    obj.capital()
    obj.language()
    obj.type()
obj_ind = India()
obj_usa = USA()
func(obj_ind)
func(obj_usa)
```

Kết quả in ra là:

```
New Delhi is the capital of India.
Hindi is the most widely spoken language of India.
India is a developing country.
Washington, D.C. is the capital of USA.
English is the primary language of USA.
USA is a developed country.
```

6.Class variable và Static variable trong Python

Class variable (biến thuộc về lớp) hoặc static variable (biến tĩnh) đều là các biến được chia sẻ cho tất cả các đối tượng của lớp. Còn Instance variable (biến thuộc về thể hiện của lớp) hoặc non-static variable (biến không tĩnh) thì là khác nhau

giữa các đối tượng khác nhau (mỗi đối tượng đều có một bản sao của các instance variable và non-static variable của lớp).

Ví dụ, giả sử ta có một Computer Science Student (sinh viên ngành khoa học máy tính) thuộc lớp CSStudent. Lớp này có thể có một biến tĩnh mang giá trị "cse" cho tất cả các đối tượng. Và lớp này cũng có thể có các biến thành viên không tĩnh như là name và roll.

Trong C++ và Java, chúng ta có thể sử dụng từ khóa static để làm cho một biến bình thường trở thành một class variable (biến thuộc về class). Những biến mà không có từ khóa static đặt ở trước thì đều là các instance variable (tức là biến thuộc về đối tượng/thể hiện của lớp đó).

Cách tiếp cận trong Python thì tương đối đơn giản, nó không yêu cầu từ khóa static. Những biến mà được gán cho một giá trị ở bên trong phần code khai báo của lớp thì đều là class variable – biến thuộc về lớp. Và những biến mà được gán cho các giá trị ở bên trong các phương thức thì đều là instance variable – biến thuộc về thể hiện/đối tượng của lớp.

Ví dụ mô tả những biến được gán cho một giá trị ở bên trong phần code khai báo của lớp thì đều là class variable – biến thuộc về lớp:

```
# -----------------------------------------------------------
#Cafedev.vn - Kênh thông tin IT hàng đầu Việt Nam
#@author cafedevn
#Contact: cafedevn@gmail.com
#Fanpage: https://www.facebook.com/cafedevn
#Instagram: https://instagram.com/cafedevn
#Twitter: https://twitter.com/CafedeVn
#Linkedin: https://www.linkedin.com/in/cafe-dev-407054199/
# -----------------------------------------------------------
# Python program to show that the variables with a value
# assigned in class declaration, are class variables
# Class for Computer Science Student
class CSStudent:
    stream = 'cse'                  # Class Variable
    def __init__(self,name,roll):
        self.name = name            # Instance Variable
        self.roll = roll            # Instance Variable
# Objects of CSStudent class
a = CSStudent('Cafedev', 1)
b = CSStudent('Nerd', 2)
```

```
print(a.stream)  # prints "cse"
print(b.stream)  # prints "cse"
print(a.name)    # prints "Geek"
print(b.name)    # prints "Nerd"
print(a.roll)    # prints "1"
print(b.roll)    # prints "2"
# Class variables can be accessed using class
# name also
print(CSStudent.stream) # prints "cse"
```

Kết quả in ra là:

```
cse
cse
Cafedev
Nerd
1
2
cse
```

7.Class method và static method trong Python

7.1.Class Method – Phương thức thuộc về lớp

Cái decorator @classmethod là một function decorator được tích hợp sẵn của Python, nó là một biểu thức mà được xử lý sau khi hàm của bạn được định nghĩa. Kết quả của phép xử lý biểu thức đó sẽ được phản chiếu lên phần code định nghĩa hàm của bạn. Một class method (phương thức/hàm thuộc về lớp nào đó) mà nhận vào một class khác làm đối số ngầm định đầu tiên, sẽ giống như một instance method (phương thức/hàm thuộc về một thể hiện/đối tượng của lớp nào đó) nhận vào một instance.

1. Cú pháp

```
class C(object):
    @classmethod
    def fun(cls, arg1, arg2, ...):
```

Trong đó:

– **fun**: Là hàm mà cần chuyển đổi thành một class method

– **returns**: một class method dành cho hàm này sẽ được trả về.

Một class method là một method – phương thức mà được gắn với class, mà không phải là gắn với đối tượng của class.

Các class methods có quyền truy cập đến trạng thái của class, bởi vì chúng đều nhận vào một tham số mà trỏ đến class, chứ không phải trỏ đến đối tượng/thể hiện.

Class method có thể thay đổi trạng thái của một class, và sự thay đổi này có thể tác động tới tất cả các thể hiện của class này. Ví dụ, nó có thể thay đổi một biến của class, trong khi biến này được sử dụng bởi tất cả các thể hiện của class.

7.2.Static method – Phương thức tĩnh

Static method không nhận vào đối số đầu tiên ngầm định nào.

1. Cú pháp

```
class C(object):
    @staticmethod
    def fun(arg1, arg2, ...):
        ...
```

Trong đó:

returns: Một static method dành cho hàm fun

Một static method là một method – phương thức mà được gắn với class, chứ không phải gắn với đối tượng của class.

Static method không thể truy cập hay sửa đổi trạng thái của class.

Static method hiện diện bên trong class bởi vì nó có ý nghĩa đối với method – phương thức có mặt trong class.

7.3.Class method và Static method

– Class method nhận vào cls làm tham số đầu tiên, trong khi static method không cần tới tham số đặc biệt nào.

– Class method có thể truy cập hoặc sửa đổi trạng thái của class, trong khi static method không thể truy cập tới hay sửa đổi nó.

– Nói chung, static method không biết gì về trạng thái của class. Chúng là những method – phương thức thuộc kiểu tiện ích, nhận vào một số tham số và làm việc trên các tham số đó. Mặt khác, class method phải có được một class làm tham số truyền vào cho nó.

– Chúng ta sử dụng cái decorator @classmethod trong Python để tạo ra một class method (phương thức/hàm thuộc về lớp) và sử dụng decorator @staticmethod để tạo ra một static method (phương thức/hàm tĩnh) trong Python.

7.4.Khi nào nên sử dụng cái nào?

Chúng ta thường sử dụng class method để tạo ra các factory methods. Các factory methods sẽ trả về đối tượng của lớp (tương tự như một hàm constructor) cho các trường hợp sử dụng khác nhau.

Chúng ta sẽ thường sử dụng các static methods để tạo ra các hàm tiện ích (utility functions).

7.5.Làm thế nào để khai báo class method và static method?

Để khai báo một class method trong Python, chúng ta sẽ sử dụng decorator @classmethod, và để khai báo một static method chúng ta sẽ sử dụng decorator @staticmethod.

Chúng ta cùng xem một ví dụ sau để hiểu được sự khác biệt giữa class method và static method. Giả sử, chúng ta muốn tạo ra một class Person. Lúc này, Python không hỗ trợ nạp chồng phương thức (method overloading) giống như C++ hoặc Java, vì vậy chúng ta sẽ sử dụng các class methods để tạo ra các factory methods. Trong ví dụ dưới đây, chúng ta sẽ sử dụng một class method để tạo ra một đối tượng person từ birth year (thông tin về năm sinh).

Như đã giải thích ở trên, chúng ta sẽ sử dụng các static methods để tạo ra các hàm tiện ích – utility functions. Trong ví dụ dưới đây, chúng ta sẽ sử dụng một static method để kiểm tra xem một person (người) có phải là adult (người lớn) hay không.

Đoạn chương trình Python mô tả cách sử dụng class method và static method

```
# -------------------------------------------------------------
#Cafedev.vn - Kênh thông tin IT hàng đầu Việt Nam
#@author cafedevn
#Contact: cafedevn@gmail.com
#Fanpage: https://www.facebook.com/cafedevn
#Instagram: https://instagram.com/cafedevn
#Twitter: https://twitter.com/CafedeVn
#Linkedin: https://www.linkedin.com/in/cafe-dev-407054199/
# -------------------------------------------------------------
# Python program to demonstrate
# use of class method and static method.
from datetime import date
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    # a class method to create a Person object by birth year.
    @classmethod
```

```
    def fromBirthYear(cls, name, year):
        return cls(name, date.today().year - year)
    # a static method to check if a Person is adult or not.
    @staticmethod
    def isAdult(age):
        return age > 18
person1 = Person('mayank', 21)
person2 = Person.fromBirthYear('mayank', 1996)
print person1.age
print person2.age
# print the result
print Person.isAdult(22)
```

Kết quả in ra là:

```
21
21
True
```

8.Thay đổi các biến thành viên trong Python

Chúng ta đã biết rằng không có từ khóa static trong Python. Tất cả các biến được gán giá trị bên trong phần code khai báo class đều là các class variable – biến thuộc về lớp.

Chúng ta nên cẩn thận khi thay đổi giá trị của class variable. Nếu chúng ta cố gắng thay đổi class variable bằng cách sử dụng đối tượng, một instance variable mới (hay còn gọi là non-static variable) cho đối tượng cụ thể đó sẽ được tạo ra, và cái instance variable này sẽ "đè" lên các class variables. Dưới đây là đoạn chương trình Python mô tả việc này.

```
# ------------------------------------------------------------
#Cafedev.vn - Kênh thông tin IT hàng đầu Việt Nam
#@author cafedevn
#Contact: cafedevn@gmail.com
#Fanpage: https://www.facebook.com/cafedevn
#Instagram: https://instagram.com/cafedevn
#Twitter: https://twitter.com/CafedeVn
#Linkedin: https://www.linkedin.com/in/cafe-dev-407054199/
# ------------------------------------------------------------
# Class for Computer Science Student
class CSStudent:
```

```
    stream = 'cse'      # Class Variable
    def __init__(self, name, roll):
        self.name = name
        self.roll = roll
# Driver program to test the functionality
# Creating objects of CSStudent class
a = CSStudent("cafedevn", 1)
b = CSStudent("Nerd", 2)
print "Initially"
print "a.stream =", a.stream
print "b.stream =", b.stream
# This thing doesn't change class(static) variable
# Instead creates instance variable for the object
# 'a' that shadows class member.
a.stream = "ece"
print "\nAfter changing a.stream"
print "a.stream =", a.stream
print "b.stream =", b.stream
```

Kết quả in ra là:

```
Initially
a.stream = cse
b.stream = cse
After changing a.stream
a.stream = ece
b.stream = cse
```

Chúng ta chỉ nên thay đổi class variable bằng cách sử dụng class name – tên lớp

Dưới đây là đoạn chương trình Python mô tả cách thay đổi class variable trong Python

```
# Program to show how to make changes to the
# class variable in Python
# Class for Computer Science Student
class CSStudent:
    stream = 'cse'      # Class Variable
    def __init__(self, name, roll):
        self.name = name
```

```
        self.roll = roll
# New object for further implementation
a = CSStudent("check", 3)
print "a.tream =", a.stream
# Correct way to change the value of class variable
CSStudent.stream = "mec"
print "\nClass variable changes to mec"
# New object for further implementation
b = CSStudent("carter", 4)
print "\nValue of variable steam for each object"
print "a.stream =", a.stream
print "b.stream =", b.stream
```

Kết quả in ra là:

```
a.tream = cse
Class variable changes to mec
Value of variable steam for each object
a.stream = mec
b.stream = mec
```

9.Hàm constructor trong Python

Các hàm constructors (hàm khởi tạo) thường được sử dụng để khởi tạo một đối tượng trong lập trình hướng đối tượng. Nhiệm vụ của hàm constructor là khởi tạo (gán các giá trị) cho các biến dữ liệu thành viên của lớp khi một đối tượng của lớp được tạo ra. Trong Python, phương thức __init()__ được gọi là hàm constructor và luôn được gọi khi một đối tượng được tạo ra.

Cú pháp khai báo hàm constructor:

```
def __init__(self):
    # body of the constructor
```

Các loại hàm constructor

– Hàm constructor mặc định – default constructor: Hàm constructor mặc định là hàm constructor đơn giản, không nhận vào bất kỳ đối số nào. Phần code khai báo của nó chỉ có một đối số là tham chiếu đến chính thể hiện/đối tượng đang được khởi tạo.

– Hàm constructor có tham số: Hàm constructor có tham số sẽ nhận vào đối số đầu tiên là một tham chiếu đến đối tượng/thể hiện đang được khởi tạo, chính là tham chiếu self, và phần còn lại là các đối số được cung cấp bởi lập trình viên.

Ví dụ về hàm constructor mặc định:

```
# ----------------------------------------------------------
```

```
#Cafedev.vn - Kênh thông tin IT hàng đầu Việt Nam
#@author cafedevn
#Contact: cafedevn@gmail.com
#Fanpage: https://www.facebook.com/cafedevn
#Instagram: https://instagram.com/cafedevn
#Twitter: https://twitter.com/CafedeVn
#Linkedin: https://www.linkedin.com/in/cafe-dev-407054199/
# ------------------------------------------------------------
class Cafedev:
    # default constructor
    def __init__(self):
        self.geek = "Cafedev"
    # a method for printing data members
    def print_Cafedev(self):
        print(self.cafedev)
# creating object of the class
obj = Cafedev()
# calling the instance method using the object obj
obj.print_cafedev()
```

Kết quả in ra là:

```
Cafedev
```

Ví dụ về hàm constructor có tham số:

```
class Addition:
    first = 0
    second = 0
    answer = 0
    # parameterized constructor
    def __init__(self, f, s):
        self.first = f
        self.second = s
    def display(self):
        print("First number = " + str(self.first))
        print("Second number = " + str(self.second))
        print("Addition of two numbers = " + str(self.answer))
    def calculate(self):
        self.answer = self.first + self.second
```

```
# creating object of the class
# this will invoke parameterized constructor
obj = Addition(1000, 2000)
# perform Addition
obj.calculate()
# display result
obj.display()
```

Kết quả in ra là:

```
First number = 1000
Second number = 2000
Addition of two numbers = 3000
```

10.Hàm destructor trong Python

Các hàm destructor (hàm hủy) sẽ được gọi khi một đối tượng bị hủy đi. Trong Python, các hàm destructors không quá cần thiết như trong C++, bởi vì Python đã có sẵn một garbage collector – trình thu gom bộ nhớ, giúp xử lý tự động việc quản lý bộ nhớ.

Phương thức __del__() còn được biết đến là phương thức destructor trong Python. Nó sẽ được gọi khi tất cả các tham chiếu đến đối tượng bị xóa đi, ví dụ như khi một đối tượng được garbage collector thu thập nhằm giải phóng bộ nhớ.

Cú pháp của hàm destructor trong Python:

```
def __del__(self):
  # body of destructor
```

Lưu ý: Tham chiếu đến đối tượng cũng sẽ bị xóa đi khi đối tượng này không còn được tham chiếu tới nữa, hoặc khi chương trình kết thúc.

Ví dụ 1: Dưới đây là một ví dụ đơn giản về hàm destructor. Bằng cách sử dụng từ khóa del, chúng ta sẽ xóa đi tất cả các tham chiếu của đối tượng ‘obj’, do đó hàm destructor sẽ tự động được gọi

```
# ----------------------------------------------------------
#Cafedev.vn - Kênh thông tin IT hàng đầu Việt Nam
#@author cafedevn
#Contact: cafedevn@gmail.com
#Fanpage: https://www.facebook.com/cafedevn
#Instagram: https://instagram.com/cafedevn
#Twitter: https://twitter.com/CafedeVn
#Linkedin: https://www.linkedin.com/in/cafe-dev-407054199/
# ----------------------------------------------------------
```

```
# Python program to illustrate destructor
class Employee:
    # Initializing
    def __init__(self):
        print('Employee created.')
    # Deleting (Calling destructor)
    def __del__(self):
        print('Destructor called, Employee deleted.')
obj = Employee()
del obj
```

Kết quả in ra là:

```
Employee created.
Destructor called, Employee deleted.
```

Lưu ý: Các hàm destructors sẽ được gọi sau khi chương trình đã kết thúc, hoặc khi tất cả các tham chiếu đến đối tượng bị xóa đi. Ví dụ như khi reference count – bộ đếm tham chiếu trở về 0. Còn đối với trường hợp object bị went out of scope, tức là chương trình đã chạy qua cái đoạn code chứa đối tượng rồi, thì tham chiếu tới đối tượng vẫn sẽ tồn tại, chứ không bị xóa đi, nên hàm destructor sẽ không được gọi.

Ví dụ 2: Sau đây là một ví dụ giải thích lưu ý ở bên trên. Trong ví dụ này, hàm destructor sẽ được gọi sau khi chuỗi 'Program End…' được in ra.

```
# Python program to illustrate destructor
class Employee:
    # Initializing
    def __init__(self):
        print('Employee created')
    # Calling destructor
    def __del__(self):
        print("Destructor called")
def Create_obj():
    print('Making Object...')
    obj = Employee()
    print('function end...')
    return obj
print('Calling Create_obj() function...')
obj = Create_obj()
print('Program End...')
```

Kết quả in ra là:

```
Calling Create_obj() function...
Making Object...
Employee created
function end...
Program End...
Destructor called
```

Ví dụ 3: Xét ví dụ về hàm destructor sau:

```
# ----------------------------------------------------------
#Cafedev.vn - Kênh thông tin IT hàng đầu Việt Nam
#@author cafedevn
#Contact: cafedevn@gmail.com
#Fanpage: https://www.facebook.com/cafedevn
#Instagram: https://instagram.com/cafedevn
#Twitter: https://twitter.com/CafedeVn
#Linkedin: https://www.linkedin.com/in/cafe-dev-407054199/
# ----------------------------------------------------------
# Python program to illustrate destructor
class A:
    def __init__(self, bb):
        self.b = bb
class B:
    def __init__(self):
        self.a = A(self)
    def __del__(self):
        print("die")
def fun():
    b = B()
fun()
```

Kết quả in ra là:

```
die
```

Trong ví dụ 3 ở trên, khi hàm fun() được gọi, nó sẽ tạo ta một thể hiện của class B, cái mà sẽ được truyền sang cho class A, sau đó class A lại thiết lập một tham chiếu tới class B, điều này sẽ gây ra một circular reference – tham chiếu vòng.

Nói chung, garbage collector (trình thu gom rác) của Python sẽ được sử dụng để phát hiện ra các loại cham chiếu vòng này, sau đó loại bỏ chúng. Nhưng

trong ví dụ này, việc sử dụng hàm destructor đã được tùy chỉnh sẽ đánh dấu cái tác vụ này là "không thể thu thập – uncollectable".

Đơn giản là, nó không biết được thứ tự để xóa đi các đối tượng là gì, vì vậy nó sẽ mặc kệ, cứ để chúng ở đó. Do đó, nếu các thể hiện của bạn có liên quan đến các tham chiếu vòng, thì chúng vẫn sẽ tồn tại bên trong bộ nhớ, chừng nào ứng dụng còn chạy.

11.Hàm str() và hàm repr() trong Python

Cả hai hàm str() và repr() đều được sử dụng để lấy về dạng thức kiểu string của một đối tượng

1. Ví dụ về hàm str():

```
# ------------------------------------------------------------
#Cafedev.vn - Kênh thông tin IT hàng đầu Việt Nam
#@author cafedevn
#Contact: cafedevn@gmail.com
#Fanpage: https://www.facebook.com/cafedevn
#Group: https://www.facebook.com/groups/cafedev.vn/
#Instagram: https://instagram.com/cafedevn
#Twitter: https://twitter.com/CafedeVn
#Linkedin: https://www.linkedin.com/in/cafe-dev-407054199/
#Pinterest: https://www.pinterest.com/cafedevvn/
#YouTube: https://www.youtube.com/channel/UCE7zpY_SlHGEgo67pHxqIoA/
# ------------------------------------------------------------
s = 'Hello, Cafedev.'
print str(s)
print str(2.0/11.0)
```

Kết quả in ra là:

```
Hello, Cafedev.
0.181818181818
```

2. Ví dụ về hàm repr():

```
s = 'Hello, Cafedev.'
print repr(s)
print repr(2.0/11.0)
```

Kết quả in ra là:

```
'Hello, Cafedev.'
0.18181818181818182
```

Từ kết quả đầu ra của đoạn chương trình ví dụ trên, chúng ta có thể thấy được rằng nếu chúng ta sử dụng hàm repr() để in ra chuỗi, thì chuỗi được in ra sẽ được bao bọc trong một cặp dấu nháy đơn (‘ ‘), và nếu chúng ta sử dụng hàm repr() để tính toán một giá trị cụ thể, thì ta sẽ thu được kết quả có độ chính xác cao hơn so với khi sử dụng hàm str().

Có thể liệt kê các điểm khác biệt giữa hàm str() và hàm repr() là:

– str() được sử dụng để tạo ra dữ liệu đầu ra dành cho người dùng cuối, trong khi hàm repr() được sử dụng chủ yếu cho việc debugging và phát triển phần mềm. Mục tiêu của hàm repr() là không rõ ràng, còn mục tiêu của hàm str() là cho ra một kết quả có thể đọc được, Ví dụ, nếu chúng ta nghi ngờ một số kiểu float đang tiềm tàng một lỗi nhỏ về việc làm tròn, thì hàm repr() sẽ cho chúng ta thấy được lỗi này có xảy ra hay không, trong khi hàm str() có thể sẽ không làm được điều đó.

– Hàm repr() sẽ tính toán dạng thức kiểu string “chính thức” của một đối tượng (một dạng thức mà chứa tất cả các thông tin về đối tượng), còn hàm str() thì được sử dụng để tính toán dạng thức kiểu string “không chính thức” của một đối tượng (một dạng thức hữu dụng khi cần in ra đối tượng).

– Câu lệnh print và hàm str() được tích hợp sẵn trong Python sử dụng __str__ để hiển thị dạng biểu diễn theo kiểu string của đối tượng, trong khi hàm repr() sẽ sử dụng __repr__ để hiển thị đối tượng.

Để có thể hiểu được những điều trên, chúng ta cùng theo dõi ví dụ sau:

```
import datetime
today = datetime.datetime.now()
# Prints readable format for date-time object
print str(today)
# prints the official format of date-time object
print repr(today)
```

Kết quả in ra là:

```
2016-02-22 19:32:04.078030
datetime.datetime(2016, 2, 22, 19, 32, 4, 78030)
```

Hàm str() hiển thị thông tin “date” của hôm nay theo một cách mà người dùng có thể hiển được cả ngày và giờ hiện tại là bao nhiêu.

Hàm repr() sẽ in ra dạng biểu diễn “chính thức” của một date-time object (đối tượng ngày-giờ) (nghĩa là bằng việc sử dụng dạng biểu diễn theo kiểu string “chính thức”, chúng ta có thể reconstruct – tái cấu trúc đối tượng).

3. Làm thế nào để sử dụng chúng bên trong các class do người dùng tự định nghĩa?

Một class do người dùng tự định nghĩa cũng nên có một hàm __repr__ nếu chúng ta muốn có được các thông tin cụ thể nhằm phục vụ cho việc debugging. Và nếu

bạn nghĩ rằng việc có được một phiên bản thông tin biểu diễn theo kiểu string sẽ hữu dụng cho người dùng, thì chúng ta nên tạo ra một hàm __str__ nữa.

Dưới đây là đoạn chương trình Python mô tả cách viết hàm__repr__ và hàm __str__ cho một class do người dùng tự định nghĩa.

```
# ------------------------------------------------------------
#Cafedev.vn - Kênh thông tin IT hàng đầu Việt Nam
#@author cafedevn
#Contact: cafedevn@gmail.com
#Fanpage: https://www.facebook.com/cafedevn
#Group: https://www.facebook.com/groups/cafedev.vn/
#Instagram: https://instagram.com/cafedevn
#Twitter: https://twitter.com/CafedeVn
#Linkedin: https://www.linkedin.com/in/cafe-dev-407054199/
#Pinterest: https://www.pinterest.com/cafedevvn/
#YouTube: https://www.youtube.com/channel/UCE7zpY_SlHGEgo67pHxqIoA/
# ------------------------------------------------------------
# Python program to demonstrate writing of __repr__ and
# __str__ for user defined classes
# A user defined class to represent Complex numbers
class Complex:
    # Constructor
    def __init__(self, real, imag):
        self.real = real
        self.imag = imag
    # For call to repr(). Prints object's information
    def __repr__(self):
        return 'Rational(%s, %s)' % (self.real, self.imag)
    # For call to str(). Prints readable form
    def __str__(self):
        return '%s + i%s' % (self.real, self.imag)
# Driver program to test above
t = Complex(10, 20)
print str(t)  # Same as "print t"
print repr(t)
```

Kết quả in ra là:

```
10 + i20
```

```
Rational(10, 20)
```

12.Metaprogramming bằng các Meta-classes trong Python

Khi mới nhìn thấy thuật ngữ Metaprogramming, chắc hẳn bạn sẽ cảm thấy nó có vẻ rất mới mẻ và xa lạ, nhưng nếu bạn đã từng làm việc với các decorators hoặc các metaclasses, thì khi đó bạn đã thực hành metaprogramming rồi đó. Có thể nói rằng, trong metaprogramming thì code này sẽ điều chỉnh/xử lý code khác.

Trong bài học này chúng ta sẽ tìm hiểu về các Metaclasses, tại sao và khi nào nên sử dụng chúng, và các lựa chọn thay thế là gì. Đây là một chủ đề khá nâng cao trong Python, bạn nên nắm được hai kiến thức sau để có thể hiểu rõ hơn bài học này:

– Lập trình hướng đối tượng trong Python.

– Decorator trong Python

Lưu ý: Phần kiến thức trong bài này được áp dụng từ Python 3.3 trở lên

Nội dung chính

- 1. Metaclass
- 2. Tao ra các custom Metaclass – Metaclass tùy chỉnh
- 3. Sử dụng metaclass để giải quyết các bài toán
- 4. Khi nào nên sử dụng Metaclass

12.1.Metaclass

Trong Python, mọi thứ đều được liên kết với một số kiểu dữ liệu cụ thể. Ví dụ, nếu chúng ta có một biến chứa giá trị integer, vậy thì biến đó sẽ thuộc kiểu int. Bạn có thể biết được kiểu dữ liệu của bất cứ thành phần nào nhờ hàm type().

```
# ------------------------------------------------------------
#Cafedev.vn - Kênh thông tin IT hàng đầu Việt Nam
#@author cafedevn
#Contact: cafedevn@gmail.com
#Fanpage: https://www.facebook.com/cafedevn
#Group: https://www.facebook.com/groups/cafedev.vn/
#Instagram: https://instagram.com/cafedevn
#Twitter: https://twitter.com/CafedeVn
#Linkedin: https://www.linkedin.com/in/cafe-dev-407054199/
#Pinterest: https://www.pinterest.com/cafedevvn/
#YouTube: https://www.youtube.com/channel/UCE7zpY_SlHGEgo67pHxqIoA/
# ------------------------------------------------------------
num = 23
print("Type of num is:", type(num))
```

```
lst = [1, 2, 4]
print("Type of lst is:", type(lst))
name = "Atul"
print("Type of name is:", type(name))
```

Kết quả in ra là:

```
Type of num is: <class 'int'>
Type of lst is: <class 'list'>
Type of name is: <class 'str'>
```

Mọi kiểu dữ liệu trong Python đều được cài đặt dựa trên các lớp (class). Vì vậy, trong ví dụ trên, không giống như C hay Java mà tại đó int, char, và float là các kiểu dữ liệu chính, thì trong Python chúng đều là đối tượng, đối tượng của lớp int, hoặc đối tượng của lớp str. Vì vậy chúng ta có thể tạo ra một kiểu dữ liệu mới bằng cách tạo ra một lớp (class) của kiểu dữ liệu đó. Ví dụ, chúng ta có thể tạo ra một kiểu dữ liệu Student bằng cách tạo ra lớp Student:

```
class Student:
    pass
stu_obj = Student()
# Print type of object of Student class
print("Type of stu_obj is:", type(stu_obj))
```

Kết quả in ra là:

```
Type of stu_obj is: <class '__main__.Student'>
```

Bản thân một class cũng chính là một đối tượng, và cũng giống như bất kỳ đối tượng nào khác, nó là một thể hiện của một thứ gọi là **Metaclass**. Một class đặc biệt có tên là **type** đã tạo ra các đối tượng Class này. class **type** là một **metaclass** mặc định chịu trách nhiệm tạo ra các class. Ví dụ, trong đoạn chương trình ở phía trên nếu chúng ta thử tìm hiểu kiểu dữ liệu của class Student, ta sẽ thấy được nó thuộc kiểu **type**.

```
class Student:
    pass
# Print type of Student class
print("Type of Student class is:", type(Student))
```

Kết quả in ra là:

```
Type of Student class is: <class 'type'>
```

Bởi vì các class cũng đều là các đối tượng (object), nên chúng có thể được chỉnh sửa theo cùng một cách. Chúng ta có thể thêm hoặc bớt đi các trường hoặc các phương thức bên trong một class theo cùng một cách mà chúng ta đã làm với các đối tượng khác. Ví dụ:

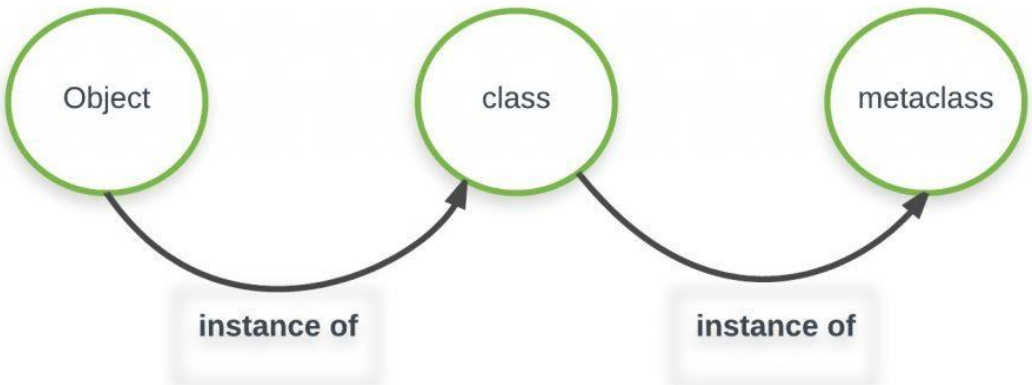
```
# ------------------------------------------------------------
#Cafedev.vn - Kênh thông tin IT hàng đầu Việt Nam
```

```
#@author cafedevn
#Contact: cafedevn@gmail.com
#Fanpage: https://www.facebook.com/cafedevn
#Group: https://www.facebook.com/groups/cafedev.vn/
#Instagram: https://instagram.com/cafedevn
#Twitter: https://twitter.com/CafedeVn
#Linkedin: https://www.linkedin.com/in/cafe-dev-407054199/
#Pinterest: https://www.pinterest.com/cafedevvn/
#YouTube: https://www.youtube.com/channel/UCE7zpY_SlHGEgo67pHxqIoA/
# ------------------------------------------------------------
# Defined class without any
# class methods and variables
class test:pass
# Defining method variables
test.x = 45
# Defining class methods
test.foo = lambda self: print('Hello')
# creating object
myobj = test()
print(myobj.x)
myobj.foo()
```

Kết quả in ra là:

```
45
Hello
```

Có thể tóm tắt quá trình này như sau: Các metaclass tạo ra các class, rồi các class lại tạo ra các object (đối tượng).



Metaclass chịu trách nhiệm tạo ra các classes, vì vậy chúng ta có thể tự viết được metaclass tùy chỉnh của riêng mình để thay đổi cách mà các classes được tạo ra,

bằng cách thực hiện các hành động bổ sung, hoặc injecting code – tiêm mã. Thường thì chúng ta sẽ không cần đến các metaclasses tùy chỉnh, nhưng đôi khi chúng khá cần thiết.

Có những bài toàn mà cả các giải pháp dựa trên metaclass và dựa trên các non-metaclass đều khả dụng (thường đơn giản hơn), nhưng trong một số trường hợp, chỉ các metaclass mới có thể giải quyết được vấn đề. Chúng ta sẽ thảo luận vấn đề đó trong bài này.

12.2.Tạo ra các custom Metaclass – Metaclass tùy chỉnh

Để tạo ra một metaclass tùy chỉnh, metaclass tùy chỉnh của chúng ta cần kế thừa từ metaclass **type**, và thường ghi đè (override) các phương thức sau:

– __new__(): Phương thức này được gọi trước phương thức __init()__. Nó tạo ra một đối tượng và trả về nó. Chúng ta có thể ghi đè (override) lại phương thức này để kiểm soát cách mà đối tượng được tạo ra.

– __init__(): Phương thức này sẽ khởi tạo cái đối tượng đã được tạo ra, cái mà được truyền vào làm tham số.

Chúng ta có thể tạo ra các classes bằng cách sử dụng trực tiếp hàm type(). Nó có thể được gọi đến theo những cách sau:

– Khi được gọi chỉ với một đối số, nó sẽ trả về kiểu dữ liệu, chúng ta đã từng thấy điều này trong các ví dụ trên.

– Khi được gọi với 3 tham số, nó sẽ tạo ra một class. Các đối số được truyền vào cho hàm type() trong trường hợp này là:

+ Class name – tên lớp

+ Tuple trong đó các base classes (lớp cha. lớp cơ sở) được kế thừa bởi các class

+ Class Dictionary: Nó phục vụ như một namespace cục bộ dành cho class, được tổ chức bởi các class methods (phương thức thuộc về class) và các biến.

Cùng xét ví dụ sau:

```
# ------------------------------------------------------------
#Cafedev.vn - Kênh thông tin IT hàng đầu Việt Nam
#@author cafedevn
#Contact: cafedevn@gmail.com
#Fanpage: https://www.facebook.com/cafedevn
#Group: https://www.facebook.com/groups/cafedev.vn/
#Instagram: https://instagram.com/cafedevn
#Twitter: https://twitter.com/CafedeVn
#Linkedin: https://www.linkedin.com/in/cafe-dev-407054199/
#Pinterest: https://www.pinterest.com/cafedevvn/
#YouTube: https://www.youtube.com/channel/UCE7zpY_SlHGEgo67pHxqIoA/
```

```
# ----------------------------------------------------------
def test_method(self):
    print("This is Test class method!")
# creating a base class
class Base:
    def myfun(self):
        print("This is inherited method!")
# Creating Test class dynamically using
# type() method directly
Test = type('Test', (Base, ), dict(x="atul", my_method=test_method))
# Print type of Test
print("Type of Test class: ", type(Test))
# Creating instance of Test class
test_obj = Test()
print("Type of test_obj: ", type(test_obj))
# calling inherited method
test_obj.myfun()
# calling Test class method
test_obj.my_method()
# printing variable
print(test_obj.x)
```

Kết quả in ra là:

```
Type of Test class:  <class 'type'>
Type of test_obj:  <class '__main__.Test'>
This is inherited method!
This is Test class method!
atul
```

Nhưng nếu chúng ta muốn tạo ra một metaclass mà không trực tiếp sử dụng hàm type() thì sao? Trong ví dụ sau đây, chúng ta sẽ tạo ra một metaclass tên là MultiBases, nó sẽ kiểm tra xem class đang được tạo ra có kế thừa từ nhiều hơn một base class (lớp cha) hay không. Nếu đúng như vậy, nó sẽ đưa ra một thông báo lỗi.

```
# our metaclass
class MultiBases(type):
    # overriding __new__ method
    def __new__(cls, clsname, bases, clsdict):
        # if no of base classes is greator than 1
```

```
        # raise error
        if len(bases)>1:
            raise TypeError("Inherited multiple base classes!!!")
        # else execute __new__ method of super class, ie.
        # call __init__ of type class
        return super().__new__(cls, clsname, bases, clsdict)
# metaclass can be specified by 'metaclass' keyword argument
# now MultiBase class is used for creating classes
# this will be propagated to all subclasses of Base
class Base(metaclass=MultiBases):
    pass
# no error is raised
class A(Base):
    pass
# no error is raised
class B(Base):
    pass
# This will raise an error!
class C(A, B):
    pass
```

Kết quả in ra là:

```
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
  File "<stdin>", line 8, in __new__
TypeError: Inherited multiple base classes!!!
```

12.3.Sử dụng metaclass để giải quyết các bài toán

Có một số bài toán/vấn đề có thể được giải quyết bởi các decorators (một cách đơn giản), cũng như bởi các metaclasses. Nhưng vẫn tồn tại một vài bài toán/vấn đề mà kết quả của nó chỉ có thể đạt được nhờ các metaclasses. Giả dụ như vấn đề lặp lại các đoạn code tương tự nhau trong khi lập trình.

Để debug các class methods (các phương thức thuộc về lớp), chúng ta sẽ code sao cho, mỗi khi class method nào thực thi, sẽ in ra tên đầy đủ của nó, trước khi thực thi nó.

Có lẽ, giải pháp đầu tiên mà chúng ta đều nghĩ đến chính là sử dụng các method decorators (các decorators dành cho phương thức), dưới đây là code ví dụ cho giải pháp này:

```
# -------------------------------------------------------------
```

```
#Cafedev.vn - Kênh thông tin IT hàng đầu Việt Nam
#@author cafedevn
#Contact: cafedevn@gmail.com
#Fanpage: https://www.facebook.com/cafedevn
#Group: https://www.facebook.com/groups/cafedev.vn/
#Instagram: https://instagram.com/cafedevn
#Twitter: https://twitter.com/CafedeVn
#Linkedin: https://www.linkedin.com/in/cafe-dev-407054199/
#Pinterest: https://www.pinterest.com/cafedevvn/
#YouTube: https://www.youtube.com/channel/UCE7zpY_SlHGEgo67pHxqIoA/
# ------------------------------------------------------------
from functools import wraps
def debug(func):
    '''decorator for debugging passed function'''
    @wraps(func)
    def wrapper(*args, **kwargs):
        print("Full name of this method:", func.__qualname__)
        return func(*args, **kwargs)
    return wrapper
def debugmethods(cls):
    '''class decorator make use of debug decorator
       to debug class methods '''
    # check in class dictionary for any callable(method)
    # if exist, replace it with debugged version
    for key, val in vars(cls).items():
        if callable(val):
            setattr(cls, key, debug(val))
    return cls
# sample class
@debugmethods
class Calc:
    def add(self, x, y):
        return x+y
    def mul(self, x, y):
        return x*y
    def div(self, x, y):
```

```
        return x/y
mycal = Calc()
print(mycal.add(2, 3))
print(mycal.mul(5, 2))
```

Kết quả in ra là:

```
Full name of this method: Calc.add
5
Full name of this method: Calc.mul
10
```

Giải pháp này hoạt động ổn, nhưng vẫn có một vấn đề, sẽ thế nào nếu chúng ta muốn áp dụng cái method decorator này cho tất cả các subclasses (các lớp con) mà kế thừa từ class **Calc**. Trong trường hợp đó, chúng ta phải áp dụng một cách riêng rẽ cái method decorator cho từng subclass (lớp con) như khi chúng ta đã làm với class **Calc**.

Tuy nhiên, vấn đề ở đây là nếu chúng ta có rất nhiều subclasses, thì việc thêm decorator cho từng subclass riêng biệt hết sức bất tiện. Nếu chúng ta biết trước được rằng mỗi subclass đều phải có cái debug property – thuộc tính dành cho việc gỡ lỗi phần mềm này, thì chúng ta tốt hơn hết nên hướng đến giải pháp dựa trên metaclass.

Hãy xem giải pháp dựa trên metaclass ngay bên dưới đây, ý tưởng ở đây là các classes sẽ được tạo ra một cách bình thường, rồi sau đó ngay lập tức được đói gói bởi debug method decorator (cái decorator dành cho hàm, để phục vụ cho việc debug).

```
from functools import wraps
def debug(func):
    '''decorator for debugging passed function'''
    @wraps(func)
    def wrapper(*args, **kwargs):
        print("Full name of this method:", func.__qualname__)
        return func(*args, **kwargs)
    return wrapper
def debugmethods(cls):
    '''class decorator make use of debug decorator
       to debug class methods '''
    for key, val in vars(cls).items():
        if callable(val):
            setattr(cls, key, debug(val))
    return cls
```

```
class debugMeta(type):
    '''meta class which feed created class object
       to debugmethod to get debug functionality
       enabled objects'''
    def __new__(cls, clsname, bases, clsdict):
        obj = super().__new__(cls, clsname, bases, clsdict)
        obj = debugmethods(obj)
        return obj
# base class with metaclass 'debugMeta'
# now all the subclass of this
# will have debugging applied
class Base(metaclass=debugMeta):pass
# inheriting Base
class Calc(Base):
    def add(self, x, y):
        return x+y
# inheriting Calc
class Calc_adv(Calc):
    def mul(self, x, y):
        return x*y
# Now Calc_adv object showing
# debugging behaviour
mycal = Calc_adv()
print(mycal.mul(2, 3))
```

Kết quả in ra là:

```
Full name of this method: Calc_adv.mul
6
```

12.4.Khi nào nên sử dụng Metaclass

Hầu hết thời gian chúng ta sẽ không sử dụng các metaclasses, chúng giống như là ma thuật đen vậy, và thường được coi là một thứ gì đó hết sức phức tạp, nhưng vẫn có một số trường hợp mà chúng ta sử dụng đến các metaclasses, đó là:

– Như chúng ta đã thấy trong ví dụ trên, các metaclasses sẽ lan truyền từ trên xuống dưới trong hệ thống phân cấp thừa kế (inheritance hierachy). Do đó nó cũng sẽ ảnh hưởng đến tất cả các lớp con. Nếu gặp trường hợp như vậy, chúng ta nên sử dụng các metaclasses.

– Khi chúng ta muốn thay đổi các classes một cách tự động, khi nó được tạo ra.

– Nếu bạn là một API developer, bạn có thể sử dụng các metaclasses.

Và có một trích dẫn của ông Tim Peters như sau:

"Metaclasses là loại ma thuật sâu sắc mà 99% người dùng không bao giờ nên lo lắng về chúng. Nếu bạn phân vân liệu rằng mình có cần tới chúng hay không, điều đó có nghĩa là bạn không cần tới chúng đâu (người mà thật sự cần tới chúng sẽ biết chắc chắn rằng họ cần chúng, mà không cần phải tìm kiếm câu trả lời cho lý do vì sao".

13.Các thuộc tính của Lớp và Các thuộc tính của Thể hiện trong Python

13.1.Các thuộc tính của Lớp

Các thuộc tính của lớp (class attributes) thuộc về chính bản thân của lớp, và chúng sẽ được chia sẻ cho tất cả các thể hiện (đối tượng) của lớp này. Những thuộc tính như vậy được định nghĩa trong phần thân của lớp và thường nằm ở phía trên cùng, để dễ đọc, dễ theo dõi.

Ví dụ về các thuộc tính của lớp:

```
# ------------------------------------------------------------
#Cafedev.vn - Kênh thông tin IT hàng đầu Việt Nam
#@author cafedevn
#Contact: cafedevn@gmail.com
#Fanpage: https://www.facebook.com/cafedevn
#Group: https://www.facebook.com/groups/cafedev.vn/
#Instagram: https://instagram.com/cafedevn
#Twitter: https://twitter.com/CafedeVn
#Linkedin: https://www.linkedin.com/in/cafe-dev-407054199/
#Pinterest: https://www.pinterest.com/cafedevvn/
#YouTube: https://www.youtube.com/channel/UCE7zpY_SlHGEgo67pHxqIoA/
# ------------------------------------------------------------
# Write Python code here
class sampleclass:
    count = 0     # class attribute
    def increase(self):
        sampleclass.count += 1
# Calling increase() on an object
s1 = sampleclass()
s1.increase()
print(s1.count)
```

```
# Calling increase on one more
# object
s2 = sampleclass()
s2.increase()
print(s2.count)
print(sampleclass.count)
```

Kết quả in ra là:

```
1
2
2
```

13.2.Các thuộc tính của thể hiện

Không giống như các thuộc tính của lớp, các thuộc tính của thể hiện không được chia sẻ cho các đối tượng. Mỗi đối tượng sẽ sở hữu một bản sao các thuộc tính của thể hiện (còn trong trường hợp các thuộc tính của lớp thì tất cả các đối tượng đều tham chiếu đến một bản sao duy nhất).

Để liệt kê các thuộc tính của một thể hiện/đối tượng, chúng ta có hai hàm:

– vars(): Hàm này sẽ hiển thị các thuộc tính của một thể hiện dưới dạng một dictionary (kiểu mảng gồm các phần từ là một cặp key-value).

– dir(): Hàm này sẽ hiển thị nhiều thuộc tính hơn so với hàm vars(), bởi vì nó không bị giới hạn trong phạm vi của instance – thể hiện của lớp. Hàm này sẽ hiển thị các thuộc tính của lớp, ngoài ra, nó cũng hiển thị cả các thuộc tính của các lớp cha mà lớp hiện tại đang kề thừa.

Đoạn chương trình Python dưới đây sẽ mô tả ví dụ về các thuộc tính của thể hiện:

```
# Python program to demonstrate
# instance attributes.
class emp:
    def __init__(self):
        self.name = 'xyz'
        self.salary = 4000
    def show(self):
        print(self.name)
        print(self.salary)
e1 = emp()
print("Dictionary form :", vars(e1))
print(dir(e1))
```

Kết quả in ra là:

```
Dictionary form :{'salary': 4000, 'name': 'xyz'}
```

```
['__doc__', '__init__', '__module__', 'name', 'salary', 'show']
```

14.Reflection trong Python

Reflection đề cập đến việc code có khả năng kiểm tra các thuộc tính, của những đối tượng mà có thể được truyền vào làm tham số cho một hàm. Ví dụ, nếu chúng ta viết type(obj) thì Python sẽ trả về một đối tượng mà đại diện cho kiểu dữ liệu của obj.

Bằng cách sử dụng reflection, chúng ta có thể viết được một hàm đệ quy dùng để đảo ngược các strings, lists, hay bất kỳ kiểu dữ liệu dạng danh sách có hỗ trợ cắt và nối phần tử nào khác. Nếu một obj là một tham chiếu đến một string thì Python sẽ trả về đối tượng thuộc kiểu str. Hơn nữa, nếu chúng ta viết hàm str(), chúng ta sẽ nhận được một string là chuỗi rỗng (empty string). Nói cách khác, việc viết hàm str() sẽ giống với việc viết “”. Tương tự, việc viết hàm list() là giống với việc viết [].

Dưới đây là đoạn chương trình Python mô tả cách sử dụng reflection:

```
# -----------------------------------------------------------
#Cafedev.vn - Kênh thông tin IT hàng đầu Việt Nam
#@author cafedevn
#Contact: cafedevn@gmail.com
#Fanpage: https://www.facebook.com/cafedevn
#Group: https://www.facebook.com/groups/cafedev.vn/
#Instagram: https://instagram.com/cafedevn
#Twitter: https://twitter.com/CafedeVn
#Linkedin: https://www.linkedin.com/in/cafe-dev-407054199/
#Pinterest: https://www.pinterest.com/cafedevvn/
#YouTube: https://www.youtube.com/channel/UCE7zpY_SlHGEgo67pHxqIoA/
# -----------------------------------------------------------
# Python program to illustrate reflection
def reverse(seq):
    SeqType = type(seq)
    emptySeq = SeqType()
    if seq == emptySeq:
        return emptySeq
    restrev = reverse(seq[1:])
    first = seq[0:1]
    # Combine the result
    result = restrev + first
    return result
```

```
# Driver code
print(reverse([1, 2, 3, 4]))
print(reverse("HELLO"))
```

Kết quả in ra là:

```
[4,3,2,1]
OLLEH
```

Nội dung chính

- 1. Reflection-enabling functions
  - 1.1. Hàm type() và hàm isinstance() trong Python
- 1.2. Hàm Callable()
- 1.3. Dir:
- 1.4. Getattr

14.1.Reflection-enabling functions

Trong Python, các hàm Reflection-enabling functions bao gồm: type(), isinstance(), callable(), dir(), và getattr().

1.1. Hàm type() và hàm isinstance() trong Python

Ngôn ngữ lập trình Python có tích hợp sẵn một phương thức tên là type(), thường hữu dụng khi muốn tìm ra kiểu dữ liệu của một biết được sử dụng trong chương trình, tại runtime – thời điểm chương trình đang được thực thi.

Nếu một đối số đơn lẻ là đối tượng được truyền vào bên trong hàm type() của Python, thì nó sẽ trả về dữ liệu có kiểu chính là kiểu của đối tượng vừa được cung cấp làm đối số. Nếu có ba đối số được truyền vào hàm type() là name, bases, và dict, thì hàm type() sẽ trả về một kiểu dữ liệu đối tượng mới.

Cú pháp:

```
type(object)
type(name, bases, dict)
```

a) Hàm type() với một tham số duy nhất là đối tượng

– Ví dụ:

```
# Python code type() with a single object parameter
x = 5
s = "geeksforgeeks"
y = [1,2,3]
print(type(x))
print(type(s))
print(type(y))
```

– Kết quả in ra là:

```
class 'int'
class 'str'
```

```
class 'list'
```

b) Hàm type() với nhiều tham số

– Dưới đây là đoạn chương trình Python trong đó có sử dụng hàm type(), lần này hàm type() sẽ nhận vào 3 tham số là name, bases và dict:

```
# ------------------------------------------------------------
#Cafedev.vn - Kênh thông tin IT hàng đầu Việt Nam
#@author cafedevn
#Contact: cafedevn@gmail.com
#Fanpage: https://www.facebook.com/cafedevn
#Group: https://www.facebook.com/groups/cafedev.vn/
#Instagram: https://instagram.com/cafedevn
#Twitter: https://twitter.com/CafedeVn
#Linkedin: https://www.linkedin.com/in/cafe-dev-407054199/
#Pinterest: https://www.pinterest.com/cafedevvn/
#YouTube: https://www.youtube.com/channel/UCE7zpY_SlHGEgo67pHxqIoA/
# ------------------------------------------------------------
# Python code for type() with a name,
# bases and dict parameter
o1 = type('X', (object,), dict(a='Foo', b=12))
print(type(o1))
print(vars(o1))
class test:
        a = 'Foo'
        b = 12
o2 = type('Y', (test,), dict(a='Foo', b=12))
print(type(o2))
print(vars(o2))
```

– Kết quả in ra là:

```
{'b': 12, 'a': 'Foo', '__dict__': , '__doc__': None,
'__weakref__': }
{'b': 12, 'a': 'Foo', '__doc__': None}
```

Nếu bạn muốn kiểm tra kiểu dữ liệu của một đối tượng, thay vì sử dụng hàm type(), có lẽ bạn nên sử dụng hàm isinstance() thì tốt hơn. Điều là là do, bên cạnh tính năng tương tự như hàm type(), thì hàm isinstance() còn kiểm tra xem liệu rằng đối tượng được cung cấp có phải là một thể hiện của lớp con (subclass) hay không.

c) Hàm isinstance()

Hàm isinstance() sẽ kiểm tra xem đối tượng object (là đối số đầu tiên) có phải là một thể hiện hoặc là lớp con (subclass) của lớp classinfor (đối số thứ hai) hay không?

Cú pháp:

```
isinstance(object, classinfo)
The isinstance() takes two parameters:
object : object to be checked
classinfo : class, type, or tuple of classes and types
```

Hàm isinstance() sẽ nhận vào hai tham số:

object: đối tượng được kiểm tra

classinfor: Thông tin về class, kiểu dữ liệu, hoặc về tuple của các classes và kiểu dữ liệu.

Giá trị trả về:

Hàm isinstance() sẽ trả về true nếu object là một thể hiện hoặc là lớp con (subclass) của một lớp nào đó, hoặc trả về false nếu object là một phần tử của tuple. Nếu classinfor không phải là một kiểu dữ liệu class nào đó, hoặc không phải là một kiểu tuple, thì ngoại lệ TypeError sẽ xuất hiện.

Ví dụ về việc sử dụng hàm isinstance() trong Python:

```python
# Python code for  isinstance()
class Test:
    a = 5
TestInstance = Test()
print(isinstance(TestInstance, Test))
print(isinstance(TestInstance, (list, tuple)))
print(isinstance(TestInstance, (list, tuple, Test)))
```

Kết quả in ra là:

```
True
False
True
```

d) Hàm type() và hàm isinstance()

Có một lỗi cơ bản mà mọi người thường mắc phải, đó là sử dụng hàm type(), trong khi hàm isinstance() sẽ phù hợp hơn khi đó.

– Nếu bạn đang muốn kiểm tra liệu rằng một đối tượng cụ thể nào đó có phải là thuộc kiểu dữ liệu này hay không, tốt hơn là bạn nên sử dụng hàm isinstance() bởi vì nó sẽ kiểm tra liệu rằng đối tượng được truyền vào trong đối số đầu tiên có phải là thuộc kiểu dữ liệu giống với kiểu dữ liệu của đối tượng được truyền vào cho đối số thứ hai hay không. Do đó, nó rất hữu dụng trong trường hợp làm việc với các lớp có quan hệ kế thừa với nhau, và các old-style classes (các lớp theo kiểu

cũ, tuy nhiên Python 3 đã không hỗ trợ old-style class nữa), bởi vì tất cả chúng đều có thể hiện đối tượng kiểu kế thừa (legacy type object instance).

– Mặt khác, đối với hàm type(), nó chỉ đơn giản là trả về kiểu dữ liệu của một đối tượng và so sánh kết quả đó với một kiểu đối tượng khác, nếu hai đối tượng được so sánh có cùng kiểu dữ liệu với nhau thì kết quả của phép so sánh sẽ là True. Trong Python, thay vì kiểm tra kiểu dữ liệu của đối tượng, tốt hơn hết bạn nên sử dụng Duck Typing (việc kiểm tra kiểu dữ liệu được hoãn lại tới run-time, và được cài đặt bởi dynamic typing hoặc reflection)

Dưới đây là ví dụ sử dụng duck typing:

```
#Python code to illustrate duck typing
class User(object):
    def __init__(self, firstname):
        self.firstname = firstname
    @property
    def name(self):
        return self.firstname
class Animal(object):
    pass
class Fox(Animal):
    name = "Fox"
class Bear(Animal):
    name = "Bear"
# Use the .name attribute (or property) regardless of the type
for a in [User("Cafedev"), Fox(), Bear()]:
    print(a.name)
```

Kết quả in ra là:

```
Cafedev
Fox
Bear
```

Lý do tiếp theo để không sử dụng hàm type() chính là sự thiếu sót của nó trong việc không hỗ trợ cho những trường hợp sử dụng có liên quan đến quan hệ kế thừa.

Đoạn chương trình Python dưới đây sẽ mô tả sự thiếu sót trong việc không hỗ trợ những usecase có liên quan đến quan hệ kế thừa của hàm type():

```
#python code to illustrate the lack of
#support for inheritance in type()
class MyDict(dict):
    """A normal dict, that is always created with an "initial" key"""
```

```
    def __init__(self):
        self["initial"] = "some data"
d = MyDict()
print(type(d) == dict)
print(type(d) == MyDict)
d = dict()
print(type(d) == dict)
print(type(d) == MyDict)
```

Kết quả in ra là:

```
False
True
True
False
```

Lớp MyDict sở hữu tất cả các thuộc tính của một đối tượng dict, và không có bất kỳ phương thức mới nào. Nó sẽ hoạt động giống hệt như kiểu dictionary. Nhưng hàm type() sẽ không trả về kết quả như mong đợi.

Sử dụng hàm isinstance() là thích hợp hơn trong trường hợp này, vì nó sẽ cho ta kết quả như mong đợi:

Đoạn code dưới đây sẽ cho thấy khả năng hỗ trợ quan hệ kế thừa của hàm isinstance():

```
# -------------------------------------------------------------
#Cafedev.vn - Kênh thông tin IT hàng đầu Việt Nam
#@author cafedevn
#Contact: cafedevn@gmail.com
#Fanpage: https://www.facebook.com/cafedevn
#Group: https://www.facebook.com/groups/cafedev.vn/
#Instagram: https://instagram.com/cafedevn
#Twitter: https://twitter.com/CafedeVn
#Linkedin: https://www.linkedin.com/in/cafe-dev-407054199/
#Pinterest: https://www.pinterest.com/cafedevvn/
#YouTube: https://www.youtube.com/channel/UCE7zpY_SlHGEgo67pHxqIoA/
# -------------------------------------------------------------
#python code to show isintance() support
#inheritance
class MyDict(dict):
    """A normal dict, that is always created with an "initial" key"""
    def __init__(self):
```

```
        self["initial"] = "some data"
d = MyDict()
print(isinstance(d, MyDict))
print(isinstance(d, dict))
d = dict()
print(isinstance(d, MyDict))
print(isinstance(d, dict))
```

Kết quả in ra là:

```
True
True
False
True
```

14.2.Hàm Callable()

Bất cứ cái gì mà có thể được gọi đến, thì đều được coi là một cái callable. Khi sử dụng cho một đối tượng, hàm callable() sẽ xác định liệu rằng đối tượng đó thể được gọi đến hay không. Có thể làm cho một class trở nên callable (có thể gọi đến) bằng cách cung cấp cho nó một phương thức __call__(). Phương thức callable() sẽ trả về True nếu đối tượng được truyền vào là callable (có thể gọi đến), nếu không nó sẽ trả về False.

Ví dụ:

```
x = 5
def testFunction():
  print("Test")
y = testFunction
if (callable(x)):
    print("x is callable")
else:
    print("x is not callable")
if (callable(y)):
    print("y is callable")
else:
    print("y is not callable")
```

Kết quả in ra là:

```
x is not callable
y is callable
```

Ví dụ về callable khi được sử dụng trong OOP:

```
class Foo1:
  def __call__(self):
    print('Print Something')
print(callable(Foo1))
```

Kết quả in ra là:

```
True
```

14.3.Dir:

Phương thức dir() sẽ cố gắng trả về một danh sách các thuộc tính (attributes) hợp lệ của đối tượng. Nếu một đối tượng có phương thức __dir__(), thì phương thức này sẽ được gọi, và phải trả về một danh sách các thuộc tính. Nếu đối tượng không có phương thức __dir()__, thì phương thức này sẽ cố gắng tìm kiếm các thông tin từ thuộc tính __dict__ (nếu được khai báo), và đối tượng type (type object). Trong trường hợp này, danh sách được trả về từ phương thức dir() có thể sẽ không đầy đủ.

Ví dụ:

```
number = [1,2,3]
print(dir(number))
characters = ["a", "b"]
print(dir(number))
```

Kết quả in ra là:

```
>>> number = [1,2,3]
>>> print(dir(number))
['__add__', '__class__', '__contains__', '__delattr__', '__delitem__', '__dir__', '__doc__', '__eq__', '__f
ormat__', '__ge__', '__getattribute__', '__getitem__', '__gt__', '__hash__', '__iadd__', '__imul__', '__ini
t__', '__iter__', '__le__', '__len__', '__lt__', '__mul__', '__ne__', '__new__', '__reduce__', '__reduce_ex
__', '__repr__', '__reversed__', '__rmul__', '__setattr__', '__setitem__', '__sizeof__', '__str__', '__subc
lasshook__', 'append', 'clear', 'copy', 'count', 'extend', 'index', 'insert', 'pop', 'remove', 'reverse', '
sort']
>>> number = ["a", "b"]
>>> print(dir(number))
['__add__', '__class__', '__contains__', '__delattr__', '__delitem__', '__dir__', '__doc__', '__eq__', '__f
ormat__', '__ge__', '__getattribute__', '__getitem__', '__gt__', '__hash__', '__iadd__', '__imul__', '__ini
t__', '__iter__', '__le__', '__len__', '__lt__', '__mul__', '__ne__', '__new__', '__reduce__', '__reduce_ex
__', '__repr__', '__reversed__', '__rmul__', '__setattr__', '__setitem__', '__sizeof__', '__str__', '__subc
lasshook__', 'append', 'clear', 'copy', 'count', 'extend', 'index', 'insert', 'pop', 'remove', 'reverse', '
sort']
>>>
```

14.4.Getattr

Phương thức getattr() sẽ trả về giá trị của thuộc tính được đặt tên của một đối tượng. Nếu không tìm thấy, nó sẽ trả về giá trị mặc định đã được cung cấp cho phương thức này. Phương thức getattr sẽ nhận vào ba tham số là object (đối tượng), name (tên của thuộc tính muốn truy xuất giá trị), và default (giá trị mặc định được trả về nếu không tìm thấy giá trị của thuộc tính có tên là name, tham số này là tùy chọn, không bắt buộc phải có).

Ví dụ:

```
class Employee:
    salary = 25000
    company_name= "geeksforgeeks"
employee = Employee()
print('The salary is:', getattr(employee, "salary"))
print('The salary is:', employee.salary)
```

Kết quả in ra là:

```
The salary is: 25000
The salary is: 25000
```

15.Barrier Object trong Python

Barrier objects (đối tượng rào cản) trong Python được sử dụng để chờ cho một số lượng cố định các thread – luồng thực thi xong, trước khi bất kỳ thread cụ thể nào khác có thể thực thi tiếp chương trình. Mỗi thread sẽ gọi đến hàm wait() khi đạt đến barrier – rào cản. Barrier – rào cản sẽ chịu trách nhiệm cho việc theo dõi và ghi nhớ số lượng các lời gọi hàm wait(). Nếu số lượng các lời gọi hàm wait() này vượt quá số lượng các threads đã được khởi tạo cùng với barrier – rào cản, thì tất cả các threads tại thời điểm thực thi này, đều sẽ được giải phóng đồng thời, và barrier sẽ trả lại quyền tiếp tục thực thi chương trình cho các waiting threads – các luồng đang đợi để được thực thi.

Các barriers thậm chí còn có thể được sử dụng để truy cập một cách đồng bộ giữa các threads. Tuy nhiên, thường thì barrier sẽ được sử dụng để kết hợp output – đầu ra của các threads lại với nhau. Một barrier object (đối tượng rào cản) có thể được tái sử dụng nhiều lần cho cùng một số lượng chính xác các threads mà nó đã được khởi tạo ban đầu.

Nội dung chính

- 1. Khởi tạo một barrier
  - 1.1 Kiểm tra trạng thái của barrier:
- 1.2. parties: Số lượng các threads cần phải truyền vào cho barrier.
- 1.3. Hủy bỏ một barrier:
- 1.4. Khởi động lại barrier:
- 1.5. wait: Vượt qua barrier – rào cản.
  - 6. n_waiting: Số lượng các threads hiện tại đang chờ đợi trên barrier.

15.1.Khởi tạo một barrier

Một barrier có thể được khởi tạo bằng cách sử dụng lớp threading.Barrier như được thể hiện trong đoạn code ví dụ bên dưới đây. Con số nằm trong cặp dấu ngoặc đơn đại diện cho số lượng các threads mà barrier nên chờ.

Cú pháp khởi tạo barrier object trong Python:

barrier = threading.Barrier(number_of_threads, action = None, timeout = None)

Cú pháp trên sẽ tạo ra một barrier object cho number_of_threads. Tham số thứ hai action, là một callable, được gọi đến bởi một trong số các threads, khi chúng

được giải phóng. Tham số thứ ba timeout là giá trị thời gian chờ mặc định nếu không có giá trị nào được chỉ định cho phương thức wait().

```
# ------------------------------------------------------------
#Cafedev.vn - Kênh thông tin IT hàng đầu Việt Nam
#@author cafedevn
#Contact: cafedevn@gmail.com
#Fanpage: https://www.facebook.com/cafedevn
#Group: https://www.facebook.com/groups/cafedev.vn/
#Instagram: https://instagram.com/cafedevn
#Twitter: https://twitter.com/CafedeVn
#Linkedin: https://www.linkedin.com/in/cafe-dev-407054199/
#Pinterest: https://www.pinterest.com/cafedevvn/
#YouTube: https://www.youtube.com/channel/UCE7zpY_SlHGEgo67pHxqIoA/
# ------------------------------------------------------------
import threading
barrier = threading.Barrier(3)
class thread(threading.Thread):
    def __init__(self, thread_ID):
        threading.Thread.__init__(self)
        self.thread_ID = thread_ID
    def run(self):
        print(str(self.thread_ID) + "\n")
        barrier.wait()
thread1 = thread(100)
thread2 = thread(101)
thread1.start()
thread2.start()
barrier.wait()
print("Exit\n")
```

Kết quả in ra là:

```
100
101
Exit
```

Có thể kể đến một số lệnh gọi hàm phổ biến, liên quan đến lớp threading.Barrier là:

15.2.Kiểm tra trạng thái của barrier:

broken: Là một giá trị boolean, là True nếu barrier đang ở trong trạng thái broken – barrier (rào cản) đã bị phá vỡ.

Cú pháp:

```
barrier.broken
```

15.3.parties: Số lượng các threads cần phải truyền vào cho barrier.

Cú pháp:

```
barrier.parties
```

15.4.Hủy bỏ một barrier:

abort: Đưa barrier vào trong trạng thái broken – đã bị phá vỡ. Điều này sẽ làm cho bất kỳ những lời gọi nào đang có hiệu lực, hoặc những lời gọi trong tương lai tới hàm wait() bị thất bại, với lỗi BrokenBarrierError

Việc gọi tới hàm abort() trên barrier thường được sử dụng để bỏ qua các điều kiện bế tắc (conditions of deadlocking) trong khi thực thi chương trình.

Cú pháp:

```
barrier.abort()
```

15.5.Khởi động lại barrier:

reset: Trả barrier về trạng thái trống (empty state) mặc định. Bấy kỳ threads nào đang chờ trên barrier đó đều sẽ nhận được ngoại lệ BrokenBarrierError.

Cú pháp:

```
barrier.reset()
```

15.6.wait: Vượt qua barrier – rào cản.

Khi tất cả các threads đang tham gia vào barrier gọi đến hàm này, tất cả chúng đều sẽ được giải phóng đồng thời. Ngoài ra, nếu có giá trị timeout – thời gian chờ được cung cấp, giá trị timeout đó sẽ được ưu tiên sử dụng. Giá trị trả về của hàm này là một số nguyên integer nằm trong khoảng từ 0 cho đến số lượng các threads – 1, khác nhau đối với mỗi thread. Nếu lời gọi hàm này bị times out – hết thời gian chờ, trạng thái của barrier sẽ được đặt thành broke – đã bị phả vỡ. Phương thức này có thể đưa ra ngoại lệ BrokenBarrierError nếu barrier bị phá vỡ hoặc bị khởi động lại (reset) trong khi một thread đang chờ đợi.

Cú pháp:

```
barrier.wait(timeout = None)
```

6. n_waiting: Số lượng các threads hiện tại đang chờ đợi trên barrier.

Cú pháp:

```
barrier.n_waiting
```

Thông thường, khi một barrier object được reset – khởi động lại, hoặc bị broken – phá vỡ, ngoại lệ BrokenBarrierError sẽ được đưa ra.

Dưới đây là đoạn chương trình ví dụ, mô tả cách các barriers được sử dụng trong Python:

```
# program to demonstrate
# barriers in python
import threading
barrier = threading.Barrier(3)
class thread(threading.Thread):
    def __init__(self, thread_ID):
        threading.Thread.__init__(self)
        self.thread_ID = thread_ID
    def run(self):
        print(str(self.thread_ID) + "\n")
        print("Parties = " + str(barrier.parties) + "\n")
        print("n_waiting = " + str(barrier.n_waiting) + "\n")
        barrier.wait()
thread1 = thread(100)
thread2 = thread(101)
thread1.start()
thread2.start()
barrier.wait()
print(str(barrier.broken) + "\n")
barrier.reset()
print("n_waiting after reset = " + str(barrier.n_waiting))
barrier.abort()
print("End")
```

Kết quả in ra là:

```
100
101
Parties = 3
Parties = 3
```

```
n_waiting = 1
n_waiting = 1
False
n_waiting after reset = 0
End
```

16.Timer objects trong Python

Tuy nhiên, khoảng thời gian được khởi tạo cho timer có thể sẽ không phải là tức thời, khi hành động thực sự được thực hiện bởi trình thông dịch, bởi vì việc thực sự lập lịch cho thread tương ứng của timer object là trách nhiệm của thread scheduler – trình lập lịch cho các luồng

Timer là một sub-class (lớp con) của class Thread được định nghĩa trong Python. Nó được khởi động bằng cách gọi đến hàm start() tương ứng với timer.

16.1.Tạo ra một đối tượng Timer

Cú pháp:

```
threading.Timer(interval, function, args = None, kwargs = None)
```

Cú pháp trên sẽ tạo ra một timer, timer này sẽ khởi chạy hàm function với các đối số args và các đối số từ khóa (keyword arguments) kwargs, sau khi khoảng thời gian interval (đơn vị thời gian ở đây được dùng là giây) đã trôi qua. Nếu args là None (chính à giá trị mặc định của nó), thì một danh sách trống (empty list) sẽ được sử dụng. Nếu kwargs là None (chính là giá trị mặc định của nó), thì một từ điển trống (empty dict) sẽ được sử dụng.

Dưới đây là đoạn chương trình Python mô tả cách sử dụng timer objects trong Python:

```
# ------------------------------------------------------------
#Cafedev.vn - Kênh thông tin IT hàng đầu Việt Nam
#@author cafedevn
#Contact: cafedevn@gmail.com
#Fanpage: https://www.facebook.com/cafedevn
#Group: https://www.facebook.com/groups/cafedev.vn/
#Instagram: https://instagram.com/cafedevn
#Twitter: https://twitter.com/CafedeVn
#Linkedin: https://www.linkedin.com/in/cafe-dev-407054199/
#Pinterest: https://www.pinterest.com/cafedevvn/
#YouTube: https://www.youtube.com/channel/UCE7zpY_SlHGEgo67pHxqIoA/
# ------------------------------------------------------------
```

```
# Program to demonstrate
# timer objects in python
import threading
def gfg():
    print("Cafedev\n")
timer = threading.Timer(2.0, gfg)
timer.start()
print("Exit\n")
```

Kết quả in ra là:

```
Exit
Cafedev
```

Giải thích: Đoạn chương trình trên sẽ lập lịch cho hàm gfg() chạy sau khoảng thời gian là 5 giây, kể từ khi hàm start() được gọi.

16.2.Hủy đi một timer

Cú pháp:

```
timer.cancel()
```

Câu lệnh này sẽ dừng timer lại, và hủy việc thực thi hành động mà timer này đang thực hiện. Câu lệnh này sẽ chỉ làm việc nếu timer vẫn đang ở trong waiting stage – giai đoạn chờ đợi.

Dưới đây là đoạn chương trình Python mô tả cách hủy đi một (đối tượng) timer

```
# Program to cancel the timer
import threading
def gfg():
    print("GeeksforGeeks\n")
timer = threading.Timer(5.0, gfg)
timer.start()
print("Cancelling timer\n")
timer.cancel()
print("Exit\n")
```

Kết quả in ra là:

```
Cancelling timer
Exit
```

17.Garbage Collection trong Python

Việc cấp phát và thu hồi bộ nhớ trong Python được thực hiện tự động. Người dùng sẽ không cần thiết phải tự mình cấp phát hay thu hồi bộ nhớ bằng cách sử dụng

các câu lệnh xử lý bộ nhớ động, nhưng trong các ngôn ngữ chẳng hạn như C hay C++.

Python sử dụng hai chiến lược cấp phát bộ nhớ:

– Reference counting (đếm số tham chiếu)

– Garbage collection (trình thu gom bộ nhớ)

Trước phiên bản Python 2.0, trình thông dịch của Python (Python interpreter) chỉ sử dụng reference counting để quản lý bộ nhớ. Reference counting làm việc bằng cách đếm số lần mà một đối tượng được tham chiếu tới bởi các đối tượng khác trong hệ thống. Khi các tham chiếu đến một đối tượng không còn tồn tại nữa, reference count (số đếm tham chiếu) của đối tượng đó sẽ được giảm đi. Khi reference count giảm về 0, đối tượng sẽ được giải phóng, tức là phần bộ nhớ đã cấp phát cho đối tượng đó sẽ được thu hồi lại.

```
# Literal 9 is an object
b = 9
# Reference count of object 9
# becomes 0.
b = 4
```

Trong ví dụ trên, giá trị 9 chính là một đối tượng. Reference count của đối tượng 9

này sẽ tăng lên một đơn vị thành 1 ở dòng 1. Trong dòng 2, reference count của đối tượng 9 này sẽ trở thành 0, bởi vì tại đây, nó đã bị giảm đi. Do đó, garbage collector (trình thu gom bộ nhớ) sẽ giải phóng đối tượng này.

Một chu trình tham chiếu – reference cycle sẽ xuất hiện khi tồn tại một hoặc nhiều đối tượng tự tham chiếu đến chính bản thân nó hoặc tham chiếu tới nhau. Reference cycle thường xuất hiện khi sử dụng list, tuple, instance (các thể hiện của class), class, dictionary, và các function (hàm). Cách dễ nhất để tạo ra một reference cycle là tạo ra một đối tượng mà đối tượng này sẽ tham chiếu đến chính bản thân nó, như trong ví dụ dưới đây:

```
# ------------------------------------------------------------
#Cafedev.vn - Kênh thông tin IT hàng đầu Việt Nam
#@author cafedevn
#Contact: cafedevn@gmail.com
#Fanpage: https://www.facebook.com/cafedevn
#Group: https://www.facebook.com/groups/cafedev.vn/
#Instagram: https://instagram.com/cafedevn
#Twitter: https://twitter.com/CafedeVn
#Linkedin: https://www.linkedin.com/in/cafe-dev-407054199/
#Pinterest: https://www.pinterest.com/cafedevvn/
```

```
#YouTube: https://www.youtube.com/channel/UCE7zpY_SlHGEgo67pHxqIoA/
# ------------------------------------------------------------
def create_cycle():
    # create a list x
    x = [ ]
    # A reference cycle is created
    # here as x contains reference to
    # to self.
    x.append(x)
create_cycle()
```

Bởi vì hàm create_cycle() sẽ tạo ra một đối tượng x tham chiếu tới chính bản thân nó, nên đối tượng x sẽ không được tự động giải phóng khi hàm này trả về (returns). Điều này sẽ khiến cho phần bộ nhớ mà đối tượng x đang sử dụng bị giữ lại cho đến khi garbage collector của Python được gọi.

Nội dung chính

- 1. Các cách để tạo ra một đối tượng mà garbage collector có thể thu gom
- 2. Việc tự động thu hồi bộ nhớ đối với các chu trình tham chiếu
- 3. Thu hồi bộ nhớ thủ công

17.1.Các cách để tạo ra một đối tượng mà garbage collector có thể thu gom

```
x = []
x.append(1)
x.append(2)
# delete the list from memory or
# assigning object x to None(Null)
del x
# x = None
```

Giá trị reference count cho list x lúc này là 2. Tuy nhiên, bởi vì nó không thể được truy cập đến từ bên trong Python và không thể được sử dụng lại, nên nó được coi là garbage (rác). Trong phiên bản hiện tại của Python, list x sẽ không bao giờ được giải phóng.

17.2.Việc tự động thu hồi bộ nhớ đối với các chu trình tham chiếu

Bởi vì việc tìm ra các các chu trình tham chiếu (reference cycles) cần đến nhiều công việc tính toán, nên garbage collection – việc thu gom bộ nhớ cần phải là một hoạt động được lập lịch. Python sẽ lập trình cho việc garbage collection – việc thu gom bộ nhớ dựa trên một số ngưỡng, liên quan đến số lượng các hành động cấp phát đối tượng, và số lượng các hành động giải phóng đối tượng. Khi số lượng

các hành động cấp phát trừ đi số lượng các hành động giải phóng ra kết quả mà lớn hơn số ngưỡng, garbage collector – trình thu gom bộ nhớ sẽ hoạt động. Có thể kiểm tra số ngưỡng dành cho các đối tượng mới (đối tượng trong Python còn được gọi là các đối tượng thế hệ 0) bằng cách import module gc và truy cập đến các ngưỡng thu hồi bộ nhớ (garbage collection thresholds).

```
# loading gc
import gc
# get the current collection
# thresholds as a tuple
print("Garbage collection thresholds:",
                    gc.get_threshold())
```

Kết quả in ra là:

```
Garbage collection thresholds: (700, 10, 10)
```

Ở ví dụ này, ngưỡng mặc định của hệ thống là 700. Điều này có nghĩa là khi số lượng các hành động cấp phát bộ nhớ trừ đi số lượng các hành động giải phóng bộ nhớ mà lớn hơn 700, thì garbage collector tự động sẽ chạy. Do đó, bất kỳ phần nào trong code của bạn mà giải phóng nhiều khối bộ nhớ lớn, thì đều là những ví dụ tốt về việc thực hiện thu hồi bộ nhớ thủ công.

17.3.Thu hồi bộ nhớ thủ công

Việc gọi garbage collector một cách thủ công trong quá trình thực thi chương trình có thể là một ý tưởng về cách xử lý vấn đề bộ nhớ bị tiêu tốn bởi các chu trình tham chiếu – reference cycles. Việc thu gom bộ nhớ có thể được gọi một cách thủ công như sau:

```
# ------------------------------------------------------------
#Cafedev.vn - Kênh thông tin IT hàng đầu Việt Nam
#@author cafedevn
#Contact: cafedevn@gmail.com
#Fanpage: https://www.facebook.com/cafedevn
#Group: https://www.facebook.com/groups/cafedev.vn/
#Instagram: https://instagram.com/cafedevn
#Twitter: https://twitter.com/CafedeVn
#Linkedin: https://www.linkedin.com/in/cafe-dev-407054199/
#Pinterest: https://www.pinterest.com/cafedevvn/
#YouTube: https://www.youtube.com/channel/UCE7zpY_SlHGEgo67pHxqIoA/
# ------------------------------------------------------------
# Importing gc module
import gc
```

```
# Returns the number of
# objects it has collected
# and deallocated
collected = gc.collect()
# Prints Garbage collector
# as 0 object
print("Garbage collector: collected",
          "%d objects." % collected)
```

Trong trường hợp chỉ có một vài chu kỳ được tạo ra, việc thu gom bộ nhớ thủ công sẽ hoạt động như sau:

```
import gc
i = 0
# create a cycle and on each iteration x as a dictionary
# assigned to 1
def create_cycle():
    x = { }
    x[i+1] = x
    print x
# lists are cleared whenever a full collection or
# collection of the highest generation (2) is run
collected = gc.collect() # or gc.collect(2)
print "Garbage collector: collected %d objects." % (collected)
print "Creating cycles..."
for i in range(10):
    create_cycle()
collected = gc.collect()
print "Garbage collector: collected %d objects." % (collected)
```

Kết quả in ra là:

```
Garbage collector: collected 0 objects.
Creating cycles...
{1: {...}}
{2: {...}}
{3: {...}}
{4: {...}}
{5: {...}}
{6: {...}}
{7: {...}}
```

```
{8: {...}}
{9: {...}}
{10: {...}}
Garbage collector: collected 10 objects.
```

Có hai cách để thực hiện thu gom bộ nhớ thủ công: Thu gom bộ nhớ dựa trên thời gian (time-based garbage collection) và thu gom bộ nhớ dựa trên sự kiện (event-based garbage collection). Thu gom bộ nhớ dựa trên thời gian rất đơn giản: Garbage collector – trình thu gom bộ nhớ sẽ được gọi sau một khoảng thời gian cố định. Còn thu gom bộ nhớ dựa trên sự kiện sẽ gọi garbage collector khi xảy ra sự kiện. Ví dụ, khi một người dùng thoát khỏi ứng dụng, hoặc khi ứng dụng chuyển sang trạng thái không hoạt động (idle state).

18.Bài tập hướng đối tượng trong python

Nhằm giúp ace nâng cao kỹ năng và kiến thức lập trình Python cho người mới bắt đầu từ con số 0, hôm nay cafedev chia sẻ cho ace một số **bài tập liên quan lập trình hướng đối tượng trong python**

Mục tiêu chính – trang bị các kiến thức và kỹ thuật:

- Lập trình hướng đối tượng cơ bản: class, object
- Kế thừa – inheritance
- Lớp trừu tượng
- Abstract base class

Lưu ý: Ace vui lòng tự code xong rồi hãy tham khảo bài giải nha – để xem cách mình làm đúng hay tối ưu nhất chưa nhé.

Bài 1: Giải phương trình bậc nhất

Bài 2: Giải phương trình bậc hai

Bài 3: Viết chương trình quản lý các CD(Công Đoàn) như sau:

```
Thông tin CD:
Nhập tên CD:    Happy New Year
Nhập tên ca sỹ: ABBA
Nhập số bài hát:        7
Nhập giá thành: 185000
--- Danh sách CD: ---
# Happy New Year - ABBA - 7 - 185000
Tổng giá thành: 185000
Tiếp tục nhập: 1: Có, 0: Không  1
Nhập tên CD:    Chat với Mozart
Nhập tên ca sỹ: Mỹ Linh
Nhập số bài hát:        8
Nhập giá thành: 245000
--- Danh sách CD: ---
# Happy New Year - ABBA - 7 - 185000
# Chat với Mozart - Mỹ Linh - 8 - 245000
Tổng giá thành: 430000
Tiếp tục nhập: 1: Có, 0: Không  0
```

Người dùng lần lượt nhập thông tincủa các CD, chương trình sẽ quản lý thông tin các CD này và in ra danh sách các CD kèm theo tổng số tiền của các CD

Hướng dẫn cách xem và tải tài liệu từ trang cafedev tại đây.

Bài giải 1,2,3

Bài 4:

Yêu cầu: Xây dựng ứng dụng quản lý danh sách các giao dịch:Mô tả:Hệ thống quản lý 2 loại giao dịch:

Giao dịch vàng: Mã giao dịch, ngày giao dịch (ngày/tháng/năm), đơn giá, số lượng, loại vàngcó 3 loại 18k, 24k, 9999.

Thành tiền được tính như sau:thành tiền = số lượng * đơn giá.

Giao dịch tiền tệ: Mã giao dịch, ngày giao dịch (ngày/tháng/năm), tỷ giá (cũng là đơn giá), số lượng, loại tiền tệ có 3 loại:USD, EUR, AUD, loại giao dịch mua/bán. Thành tiền được tính như sau:

Nếu loại giao dịch là "mua"thì: thành tiền = số lượng * tỷ giá

Nếu loại giao dịch là "bán" thì: thành tiền = (số lượng * tỷ giá)* 1.05

```
Quản lý giao dịch:
Nhập mã GD:     gd001
Nhập ngày GD:   13/03/2017
Nhập số lượng:  10
Chọn loại giao dịch: 1: Vàng, 2: Tiền Tệ:       1
Chọn loại: 18k / 24k / 9999:    18k
Nhập đơn giá:   2350000
gd001 - 13/03/2017 - 18k - 10 - 2350000 - Thành tiền = 23500000
Tổng số lượng: 10
Tổng số tiền: 23500000
Bạn muốn tiếp tục giao dịch? 1: Có, 0: Không    1
Nhập mã GD:     gd002
Nhập ngày GD:   14/03/2017
Nhập số lượng:  100
Chọn loại giao dịch: 1: Vàng, 2: Tiền Tệ:       2
Chọn loại: USD / EUR / AUD:     USD
Nhập tỷ giá:    23000
Bạn mua hay bán? 1: mua, 0: bán:        1
GD mua: gd002 - 14/03/2017 - USD - 100 - 23000 - Thành tiền = 2300000
Tổng số lượng: 100
Tổng số tiền: 2300000
Bạn muốn tiếp tục giao dịch? 1: Có, 0: Không    1
```

Dựa vào mô tả trên, hãy:

Tạo lớp GiaoDichvới các thuộc tính và phương thức chung (giao dịch vàng cũng là giao dịch).

Tạo lớp GiaoDichTienTekế thừa từ lớp GiaoDichvới các thuộc tính riêng và phương thức cần thiết.

Nhập xuất danh sách các giao dịch.

Tính tổng số lượng cho từng loại.

Tính tổng thànhtiền cho từng loại.

Bài 5: Tính chu vi & diệntích các hình (abstract)

Viết chương trình tính chu vi và điện tích của một số hình như sau:

- Hình tròn
- Hình chữ nhật
- Hình tam giác